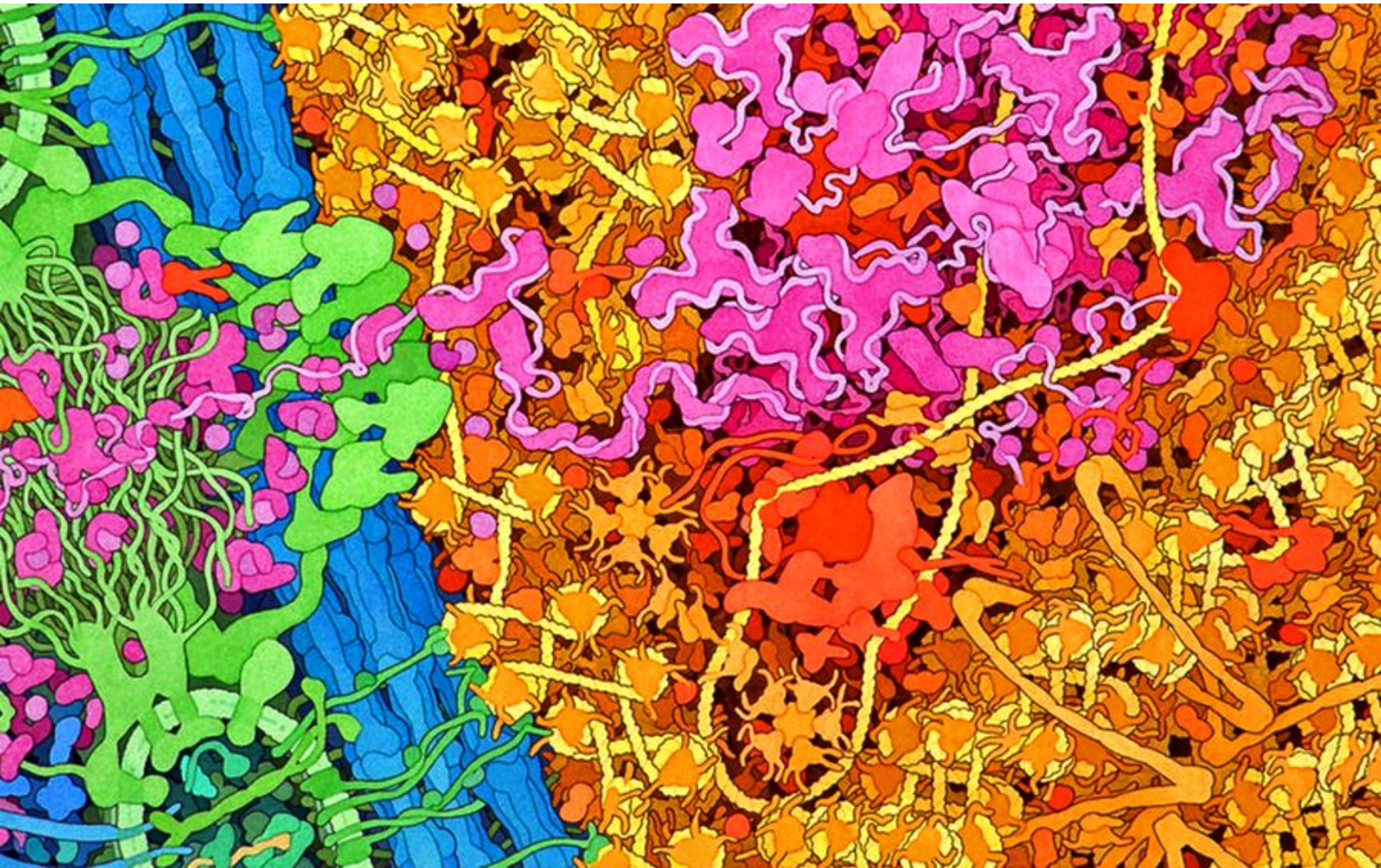


Physics of Life Processes

Course Reader

May 8, 2026

John van Noort



The complex interior of part of a cell. Image credits: David Goodsell

Contents

1	The central dogma	6
A	Background	6
B	Theory	7
C	Exercises	11
D	Test	18
2	Equipartition theorem	21
A	Background	21
B	Theory	21
C	Exercises	26
D	Test	35
3	Life at low Reynolds numbers	37
A	Background	37
B	Theory	37
C	Exercises	43
D	Test	46
4	Diffusion	49
A	Background	49
B	Theory	49
C	Exercises	56
D	Test	62
5	Entropy	64
A	Background	64
B	Theory	65
C	Exercises	73
D	Test	77
6	Reactions	80
A	Background	80
B	Theory	81
C	Exercises	88
D	Test	92

Before you begin

After reading this chapter you should be:

- able to put the course **Physics of Life Processes** in a broader perspective
- aware of the organization of the course and your expected assignments

Introduction

Crossing disciplinary borders

Life sciences have become a large part of the natural sciences. Disciplines such as pharmacy, biology, medicine, (bio-) chemistry, and even information technology may appear much more appropriate to address the processes of life than physics. However, this view has been challenged, and the current consensus (even in those fields) is that the skills that are taught in physics are an asset to life science research. In particular, the ability to reduce a problem to its bare bones and use robust mathematics to support theoretical ideas has proven invaluable.

Nevertheless, as any biologist will argue, some problems cannot be reduced to an isolated question and cannot be captured in (simple) equations. Any attempt to do so largely ignores the complexity of life. Likewise, for someone trained in physics, it may seem impossible to do rigorous science on such a complex object as a living creature. The challenge of biophysics is to find a middle way: to identify the essential ingredients that define the physics of the problem at hand, to work his/her way through the vast amount of biological data that has been disclosed by colleagues from various fields, and to filter it for its use to tackle the problem.

The contents of this course are loosely inspired by the book 'Biological Physics: Energy, Information, Life' by Philip Nelson. This book is not required, but will give you a more extensive introduction to Biophysics. A student edition is available for an attractive price.

Practical organization

Coding for data analysis and simulations

Here we will use `Python`, and in particular `Jupyter` notebooks, to get some hands-on experience with modern biophysics. It will build on the previous course `Programmiermethoden NA`, provide you with ample opportunity to build up experience with it, and it will supplement your knowledge of coding style, algorithms and the use of external modules. `Python` may not be the fastest language around, but it is the go-to language for first analysis. Likewise, `Jupyter` notebook may not provide the slickest user interface, but it is convenient and didactically it makes sense to split code into small, comprehensible cells.

You are required to use the `CoCalc` server to code your work. It provides a web implementation that allows you to work on any computer with internet, without the hassle of installing `Python`, `Jupyter`, an IDE or packages. Moreover, it keeps track of progress, and it allows you to work with multiple users simultaneously while following their input in real time. On top of that, it provides a text and a video chat option. This feature has saved us during Covid times and also when the university was suddenly closed due to bad weather conditions. You do not need to have a paid account to use it for this course (if it runs out of memory, you probably did not implement the code most optimally). This year we have a university paid subscription. In the first week we can activate it upon request. In the next weeks, it will be automatically assigned.

Collaborating with colleagues

You will be assigned a new partner each week to jointly work on both your homework and your coding assignments. You'll also submit your test assignments together. We've noticed that explaining code to a colleague is an important part of learning. Moreover, it offsets the large differences in coding experience in our student population. Finally, explaining scientific concepts is as much part of the physics curriculum as acquiring them. By teaming up with a new partner every week, your learning experience will be less affected by the skill set of a particular partner, and we aim for a distribution of tasks that will be different each week. Therefore, we *don't* allow exceptions.

Not only do students have different coding skills, some will have studied biology and/or chemistry in high school up till their final exams, others may have dropped it 2 years before. Both topics are relevant to Physics of Life processes. Similarly, some of the concepts that we will discuss have found their way into `Wiskunde D`, but not all of you have chosen this elective. Teaming up in alternating pairs is a way to smoothen some of these differences. Collaborations to acquire knowledge from different areas are 'hard-wired' into biophysics, and therefore fit well in this course.

Setup of the course

The course is organized differently from what you may be used to: the lectures are relatively short, and only provide a broad introduction to theory that you will use in the exercise classes. In the lectures we will refrain from programming

(skills): rather we want you to set up the theoretical basis and/or an intuitive framework for the problem at hand.

In the exercise classes, you and your partner will use Python for data analysis and/or simulations. Typically, you'll visualize data sets, simulate new data, and/or compare these with the theory explored in the homework. Like the homework, you'll submit a single document together at the end of the week. Attendance at the exercise classes is mandatory. During these classes, you can ask TA's for feedback on the homework and the initial problem sets. The final problem sets are tests and you and your partner will be graded on the outcome.

Finally, after 6 lectures and exercise classes, you'll work individually and at home on a problem set.

Exercise classes

Exercise classes have the following structure:

1. Group instruction (first 5-10 min.)
2. Working on that week's assignments in groups of two
3. If your group requires assistance then the teaching assistants are available during the tutorial
4. The Answers model for the problems of the exercises section will be made available one hour before the end, so you can review your answers and ask for additional feedback

You require a CoCalc account to participate in the tutorials. To create this account, use the fake email address xxxx@PoLP.2026, where xxxx are the last 4 digits of your student number.

You will find a project with weekly exercises. When you expand the + icon in front of the project, you can invite your partner (using his/her fake email address, with which he/she registered) so you can work jointly on the same document (Fig 1). Use a single **Jupyter** notebook for both your exercises and your test. A blank file with an appropriate header cell is provided in this week's project folder. When finished, you can export it as a single pdf-file and submit it in Brightspace.

The test **Jupyter** notebooks need to be submitted as a pdf-file on Brightspace before their deadlines expire. We sometimes experienced problems with saving as a pdf file. In that case save as an html, open the html file in your browser and print to pdf. Be in time to save your work as a pdf. We will be strict on the deadlines and the format in which you submit it. Since you'll be submitting 7 documents for grading, a single penalty can be balanced by one of the other problem sets. Reasonably motivated exceptions will only be granted when discussed and approved beforehand with the lecturer.

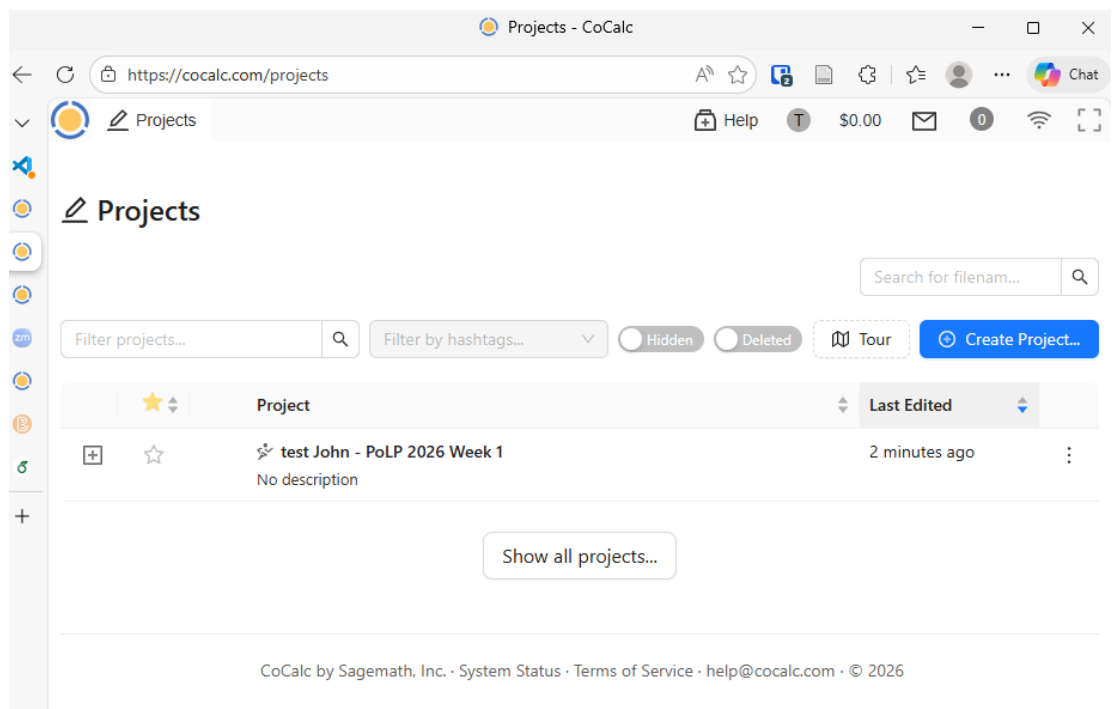


Figure 1: The CoCalc entry page lists a shared project: PoLP 2026 course that is shared with all students, TA's, and Teacher and your personal assignments. You can share your assignment with your partner by expanding the + icon in front of it, and pressing the button labeled 'Add Collaborator'. Fill out the exact username of your partner, i.e. xxxx@PoLP.2026. Only one of you needs to add the other.

Chapter 1

The central dogma

A. Background

Life as we know it

Life can be defined as the ability to reproduce. At first glance, life seems to defy the laws of physics. But that's only if you don't apply the appropriate laws. And, as we will see, those laws are generally quite different from those of Newtonian physics that you are familiar with. Mind you, we will not encounter any quantum mechanics in this course. However, we'll touch upon several topics that bothered the founders of quantum mechanics and needed to be dealt with before making progress in this field.

Size matters

As with quantum mechanics, things become interesting when the scale of the (biological) objects reduces. At some point, it becomes impossible to follow the processes that define life with the eye, or even with the best optical microscope. Typically, the processes that we are interested in happen at the scale of single molecules, which are usually 1 or 2 orders of magnitude smaller than the wavelength of light. This challenges experimental tests and validations of hypotheses about the subject.

Theory and experiment go hand in hand

Since the behavior of biological matter is generally quite complex, the development of theory on the physics of life processes has adapted to the pace set by experimental validation. Accordingly, we will try to relate the new theory to the experiments that test it. Since experimental biophysics relies heavily on advanced instrumentation (often microscopes), availability of specific biological materials, and the skills to handle them, we'll not go into the lab. Rather, we will explore experimental data sets acquired by others or use simulations to relate theory to experiment. The central question for this chapter is:

What is the composition of living matter and how does it organize itself?

B. Theory

In this section you should acquire/update:

- a generic knowledge of the molecular contents of a cell from open-source information
- 'order of magnitude' insight into the dimensions, energies, and numbers of some of the most important structures and processes in cells
- knowledge of how information that is necessary to keep cells living and dividing is stored and put into action

1 Sizes

It's good to get a grip on the dimensions of living material. You'll find the answers to these questions in many resources online or in journals or books. A good resource is Cell Biology by Numbers (<http://book.bionumbers.org>), by Ron Milo and Rob Philips.

In these problems we are looking for answers that are correct in order of magnitude; some parameters are well-defined, but others vary in time, between organisms, or for other reasons. Give a reference to the source of your data in your answers (link or author, year, and title) if the question calls for a number.

Question B.1a What is the largest organism on earth and how big is it?

Question B.1b Though viruses play a role in life, they are generally not considered to be living organisms. Why not?

Question B.1c What is the smallest organism, and how big is it?

Question B.1d Larger organisms typically consist of many cells. How many cells can be found in a typical human body?

Question B.1e Make a good guess for the size of a human and calculate the average length of a human cell. (Tip: assume a cubic cell)

2 Cells

Cells are the smallest units of life. Before genetic tools were abundant, cells could be differentiated by morphology. Though all living organisms are related and very similar processes take place inside cells, based on their morphology (next to their appearance and biological functionality) three branches were traditionally identified: bacteria, plants, and animals. The first group is referred to as prokaryotes, the latter two are eukaryotes.

Question B.2a What is the main (morphological) difference between prokaryotes and eukaryotes?

Question B.2b Name two differences in the composition of plant and animal cells.

Cells can be divided into compartments called organelles. The main organelles in plant cells are:

- nucleus
- mitochondrion
- endoplasmatic reticulum
- Golgi apparatus
- Chloroplast

Question B.2c What is the main function of each of these organelles?

3 Molecules

Organelles organize the molecules in cells into structures that support specific functionalities. Some of these molecules spontaneously assemble in functional aggregates, others are separated by membranes.

Question B.3a What is the difference between organic and inorganic molecules? Name a few in both classes.

Many of the bio-molecules that we will study are polymers: long chains consisting of small molecular units that are covalently attached to form strings. Examples of monomers are:

- (i) adenine
- (ii) glucose
- (iii) alanine
- (iv) uracil

Examples of polymers are:

- (a) RNA
- (b) DNA
- (c) protein
- (d) polysaccharide

Question B.3b Match the instances of the list of monomers to their corresponding polymers.

Question B.3c Look up the (mass) percentage of the following molecules in the cell and order them according to abundance:

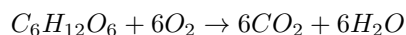
- (a) Water
- (b) small ions (K^+ , Na^+ , Li^+ , Ca^{2+} , Ni^{2+} , Cl^- , etc.)
- (c) DNA
- (d) RNA
- (e) sugars
- (f) fatty acids
- (g) proteins

4 Energies

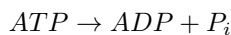
One of the laws of physics that also applies to life processes is the conservation of energy. Here we will explore the magnitudes of various biologically relevant entities. Express all requested energies in the SI unit Joule.

A common source of energy for living organisms is glucose. The energy that is released upon oxidation of glucose is used by cells to maintain other reactions. This energy can be calculated by summing the cost of breaking all the covalent bonds and subtracting the energies involved with creating new bonds.

Question B.4a How much energy is released upon oxidation of one glucose molecule, following the following reaction:



Another common reaction that releases a lot of energy is the hydrolysis of Adenosin-tri-phosphate (ATP) into Adenosin-di-phosphate (ADP) and phosphate (P_i):



Question B.4b Look up how much energy (or to be more correct free energy, but the difference will come later) is released upon hydrolysis of 1 ATP molecule under physiological conditions?

Molecules do not only have internal, chemical energy, they also have kinetic energy at finite temperatures. The amount of kinetic energy depends on temperature and equals $\frac{1}{2}k_B T$ per degree of freedom.

Question B.4c Look up Boltzmann's constant and calculate how much $1 k_B T$ is in the standard SI unit for energy at room temperature.

Question B.4d How does this relate to the energy required to break a typical covalent bond?

When at rest, a typical human produces 100 W of heat.

Question B.4e How many lumps of sugar (1 gr) would you need per hour to maintain an energy balance?

When doing strenuous labor, like for example racing up a mountain with a bike, a person can generate a power of more than 800 W. Not all this power is converted into work. Assuming an efficiency of 50 percent, half of this power is lost in heat. To efficiently lose this heat, and maintain a constant body temperature, we sweat. The evaporation of water costs energy: the latent heat of vaporization for water at room temperature is 2430 J/gr.

Question B.4f How much water needs to evaporate (and must therefore be replenished) to maintain a constant temperature under these conditions? (expressed in liters)

5 The central dogma

Next to a flow of energy that keeps life going, there is a continuous production, transport, and breakdown of materials that are required to keep life processes going. Proteins play a key role in organizing both the structure of the cell and the reactions that characterize life. Though other molecules are also involved in regulating processes and creating structures, the versatility of proteins makes them uniquely fit for an enormously wide range of functionalities.

Question B.5a How many different amino acids can be found in nature?

Question B.5b What is the average length of a (human) protein, expressed in the number of amino acids?

Question B.5c How many different proteins can be constructed give your answers to the previous questions?

Only a small fraction of the possible proteins is found in nature. The number and sequence of amino acids in each protein are encoded in DNA. The central dogma in biology states that the information flow runs from DNA, via RNA to proteins. This strictly one-directional process has been revisited more recently, but for now, we will stick to this simple description.

The conversion from DNA to proteins proceeds in two steps: (i)transcription and (ii)translation.

Question B.5d Describe, in a maximum of 3 sentences each, what these processes entail and which molecules are involved.

As opposed to proteins, DNA (and RNA) consists only of 4 types of bases. A lossless translation of a DNA sequence into a protein sequence therefore requires multiple bases per amino acid.

Question B.5e What is the minimum number of bases that are required to encode each amino acid with a unique DNA sequence?

Question B.5f What will be the average size of a human gene (a string of DNA encoding for a single protein) expressed in base pairs, given your answer to the previous questions?

Look up the total size of the human genome (expressed in base pairs) and the number of unique genes that encode proteins.

Question B.5g Using these numbers, calculate what fraction of our genome encodes for proteins.

C. Exercises

In this section you learn:

- how to obtain atomic resolution structures of bio-molecules
- basic use of the pandas library
- the difference between histograms, probability distributions, and probability density distributions
- to use probability density functions to obtain the mean, variance, and other measures of a set of parameter values

1 The protein database

James Watson and Francis Crick published the atomically detailed structure of DNA in 1953. In 1958 followed molecular structure of the first protein: haemoglobin, a protein that is responsible for carrying oxygen in red blood cells. Both were obtained by interpreting the X-ray diffraction pattern of a crystal of pure DNA or protein. These findings revolutionized our knowledge of biology which resulted in the formulation of the central dogma. Later, Nuclear Magnetic Resonance spectroscopy of dense protein solutions was added as a structural biology technique. More recently, with the development of better electronic cameras, a third technique was introduced: single-particle cryo-electron microscopy. For this method, several 10.000's noisy 2D images of a particular bio-molecule are combined to reconstruct a 3D image of the protein.

When more and more structures of proteins became available (and after scrutiny by the scientific community), all structures were deposited in a central,

open-access database called the Protein Database (pdb), founded in 1971. We can access all structures from the following website: <https://www.rcsb.org/>

Note that only in 2021 it became possible to compute protein structures from their amino acid sequence thanks to the huge progress in Machine Learning. The algorithm was trained on the data in the PDB, as well as a large library of DNA sequence data.

When working in CoCalc, create a new cell for every question and start each cell with the question number. This may be done with a comment line (i.e. '#Question1.C1b').

Question C.1a What is the total number of experimental structures that are currently available in the PDB and when was this only half the number?

Next to a lot of information on the organization of the PDB, the website <https://www.rcsb.org/> features a search function and a data visualization function. Search for the keyword hemoglobin. You will obtain a list of structures of the protein itself, as well as proteins that interact with it or are in other ways linked. Each entry has a unique 4-letter/number identification code (id).

Question C.1b What is the id of the highest resolution structure of hemoglobin? What is the resolution of this structure?

You can click on the entry and in the new page you can see more information on the structure, and you can even visualize the structure in 3D. It is fun to do this and get a good idea of the complexity of the protein.

We will not use the web interface a lot, but we want to use the data from the pdb to extract physical information from it. In addition, we will use it to refresh and expand Python skills.

When you expand the **Download Data** button, you'll be offered a lot of options. We will use the standard pdb format. Click on it and open the file that you downloaded with Notepad. You'll see a lot of text. Each 80-character line starts with a record name, and each record type has its own format. It's better to use some software to extract relevant information.

The direct link to the pdb file is <https://files.rcsb.org/download/XXXX.pdb>, in which XXXX should be replaced by the id. This is convenient because we can directly use this link in Python to download the file. For example, you can use the function `request.urlretrieve` from the library `urllib` (for more information see <https://docs.python.org/3/library/urllib.request.html>).

If you have a paid CoCalc license you can download the pdf-file directly: open a new cell in CoCalc and start with the line:

```
from urllib import request
```

Then, write a function `download_pdb(id)` that uses `id` as input, downloads the pdb file and returns the name of the downloaded file as a string. The core of this function contains a line like:

```
request.urlretrieve(url, filename)
```

Without a license you'll need to download the pdb-file manually: find the button **Download files** in the top right corner of the of the pdb web page of the structure you want, click it and select 'PDB format'. Then drag the file from your computer to the 'Files' tab in CoCalc.

Question C.1c Download the pdb-file of haemoglobin and make sure that you see it in your CoCalc project under **files**.

Although many specific software programs work with pdb files, we will write our own. We will use the library **pandas** to conveniently manipulate the data. Think of **pandas** as a spreadsheet tool: it uses 2D data in a variable type called a **dataframe**, in which the rows have a name that is referred to as the **index**. Each **column** also has a name. Below is a simple function that uses pandas to read a pdb file and only keeps the lines designated with the record name **ATOM** (See listing 1).

Question C.1d Explain what the dictionary **pdb_format** contains, and relate it to the contents of the pdb file (Tip: you can open the file in any text editor).

Question C.1e The core of the program is the **pandas** function **read_fwf()**. Look up what the letters 'fwf' represent.

We need to explicitly define the **colspecs** because the pdb file has lines with different formatting. For now, we are only interested in lines that start with the name **ATOM**. To do this, the next line in the function discards all other lines, using a procedure that you may recognize from manipulating lists or **numpy** arrays: you provide a Boolean array to keep only the elements for which the array elements are **True**. in this case, the Boolean array is generated with the code **df['record']=='ATOM'**.

Each atom in the pdb file has its own index number. We want the index of each row in the **dataframe** to correspond with the index number of the atom. This is done by the method **set_index()**, which requires the name of a column that contains the index number (in this case '**index**'). When the **set_index()** method is not used, the function **pd.read_fwf()** would assign the line number to the index of **dataframe** **pdb**.

The dataframe method **astype()** explicitly sets the data type of the selected column of the **dataframe**.

Question C.1f Explain what the two if statements do.

Question C.1g What is this procedure (selectively discarding elements in a list) called in Python?

Question C.1h Use the function **retrieve_atoms_from_pdb()** to create a **dataframe** containing the atoms of the pdb that you just downloaded. Print the first 30 records of this **dataframe** using the dataframe method **head()**.

```

import pandas as pd

def retrieve_atoms_from_pdb(filename, chainid=None, atom=None):

    pdb_format = {'record': (0, 6),
                  'index': (6, 11),
                  'atom': (12, 16),
                  'altloc': (16, 17),
                  'resname': (17, 20),
                  'chainid': (21, 22),
                  'resseq': (22, 26),
                  'icode': (26, 27),
                  'x': (30, 38),
                  'y': (38, 46),
                  'z': (46, 54),
                  'occupancy': (54, 60),
                  'tempfactor': (60, 66),
                  'element': (76, 78),
                  'charge': (78, 80)}

    df = pd.read_fwf(filename, names=pdb_format.keys(),
                     colspecs=list(pdb_format.values()))

    df = df[df['record'] == 'ATOM']
    df = df.set_index('index')
    df = df.astype({'x': float, 'y': float, 'z': float})

    if chainid is not None:
        df = df[df['chainid'] == chainid]
    if atom is not None:
        df = df[df['atom'] == atom]

    return df

```

Listing 1: Code for reading a pdb file.

(Tip: include this code to prevent `pandas.head()` from breaking up the lines: `pd.set_option('expand_frame_repr', False)`).

The columns `x`, `y`, and `z` contain the coordinates of each atom.

Question C.1i What is the unit of these coordinates?

The column `resname` contains a three-letter abbreviation.

Question C.1j What type of residues are listed here? How many different

residues are there?

(Tip: you can use the `pandas` method `unique()` to list all the unique elements in a column.)

The columns `atom` and `element` share the same first letter. Yet, the column `atom` has more letters to signify the position of the elements in the backbone of the protein.

Question C.1k There is one atom to which the variable side group of the amino acids is linked. How is this atom named in the column `atom`?

2 Histograms

Here, we will look at the contents of the `pdb` file.

Question C.2a Use the answer to the previous question to discard all entries except the central atom of each amino acid. Print the first 10 lines of the new `dataframe`.

You have learned to use the library `matplotlib` to plot data in Python. For 3D plots, you can use the following statements:

```
import matplotlib.pyplot as plt

ax = plt.axes(projection = "3d")
ax.scatter3D(x, y, z)
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_zlabel('Z axis')
plt.show()
```

Listing 2: Plot the xyz coordinates in 3D.

Many `pdb` files contain multiple proteins (and/or DNA molecules). Sometimes this is because the structure represents a complex of interacting molecules. But often this is an artifact of the crystallization method. Within the crystal, dimers or multimers of the protein can form a unit cell of the crystal, rather than a single protein. The label `chainid` identifies each individual molecule in such a unit cell. In the case of this `pdb` file, there are 2 proteins in a unit cell, labeled A and B.

Question C.2b Plot the coordinates of `chainid A` in 3D. (Tip: Don't forget to include proper axis labels.)

To quantify the dimensions of the protein better, we want to plot a histogram of the coordinates for each dimension, as shown below.

Question C.2c

Use `matplotlib` and the `numpy` function `histogram()` to reproduce this histogram in `CoCalc`. Take note of all elements that define the graph, including axis labels and legends.

(Tip 1: the function `histogram()` takes `nbins` and `range` as inputs. It is better to set these explicitly than to use the default values. Try different settings for an optimal result.)

(Tip 2: `histogram()` has two outputs: `hist` and `bin_edges`. The latter cannot directly be used as input for the x-axis for the plot (why not?). It is useful to create a new function yourself, named `center(bin_edges)` to convert `bin_edges` to the proper input. You can and should reuse this function many times.)

(Tip 3: To make the labels better readable, include `plt.rc('font', size=15)` in the code.)

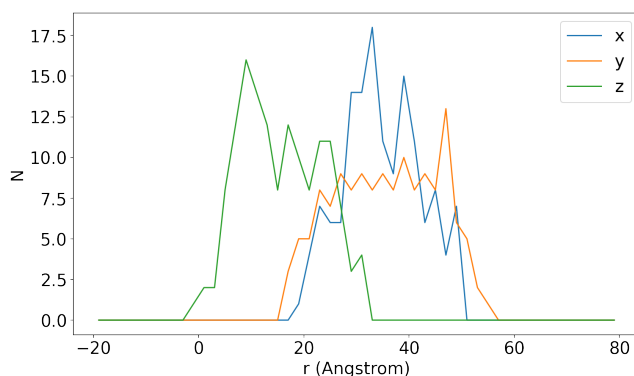


Figure 1.1: A histogram of the coordinates of the amino acids of Hemoglobin.

Though histograms are nice, and graphically depict a clear distribution of where the atoms are located in the protein, for further calculations it is often better to use a probability distribution.

Question C.2d

Plot the probability distribution for all three directions by dividing the histogram by the total number of counts. How do you calculate the probability that the x-coordinate is smaller than 40 Å if you don't have the original coordinates, but only the probability distribution?

Question C.2e

Plot the probability distribution of the y-coordinates for 3 different bin sizes. Keep the `range` parameter the same, but vary the number of bins. What happens with the probability distribution function as you take more bins?

To make the distribution independent of the chosen bin size, you can compute the probability *density* distribution by dividing the probability of each bin by the bin size.

Question C.2f Plot the probability density distribution of the y-coordinates for 3 different bin sizes and check that the magnitude of the curves is the same. What does change?

3 Moments

We would like to compute the size of the protein from the coordinates of the amino acids. Of course, the protein is not a perfect sphere, and the amino acids are positioned irregularly (though not random, because then there would not be a diffraction pattern, and its structure would be ill-defined). Here, we will use a method based on the probability density distribution to obtain the average displacement of the amino acids from the center of the protein. This will give a measure of the size of the protein, though not precisely along the surface of it. The method is introduced anyway because it is very generic and is used often to compute statistical measures.

Question C.3a Calculate the mean x, y, and z coordinate of the amino acids using values in the appropriate columns of the `dataframe`. Check that these values correspond to the approximate center of the distributions plotted in the previous question. (Tip: define a list with the names of each of the columns: `['x', 'y', 'z']` and iterate through these using the Python structure `listcomprehension`. In this way, you don't need to repeat your code and it's still faster than a `for`-loop.)

If you did it correctly, you simply calculated the mean of x by:

$$\langle x \rangle = \frac{1}{N} \sum x \quad (1.1)$$

Now let's compute the mean $\langle x \rangle$ from the probability density distribution $P(x)$:

$$\langle x \rangle = \sum x.P(x) \quad (1.2)$$

This is called the first moment of x .

Question C.3b Modify the calculation of the mean coordinates to use equation (1.2). Put the result in a list named `center_of_mass`. You don't need to use `listcomprehension` here. Check that the result is largely independent of bin size and the range of the histogram.

Question C.3c Plot the probability distribution of the distance from the center for the amino acids for all 3 dimensions.

Because we now centered the probability distribution, its mean will be 0. To quantify the average distance from the center it's better to calculate the square of the distance from the center.

Question C.3d Calculate the mean of the squared distance from the center of mass. In addition, print the square root of the result. This has the same dimension as the coordinates and can be compared to the probability distributions. Is the protein a sphere?

The same values can be calculated from the second moment of x :

$$\langle x^2 \rangle = \sum x^2 \cdot P(x) \quad (1.3)$$

Question C.3e Plot the probability distribution of the square of the distance from the center of mass for all three directions. Adjust the range of the histogram to get appropriate values. Then compute the root mean squared (RMS) distance to the center of mass using equation (1.3). What happens if you make the range of the histogram too small?

The RMS value underestimates the dimensions of the protein, as you can see in the histograms. We are not interested in the distance of all the amino acids from the center of mass, but mainly the outer amino acids. A reasonable approximation for the radius is to multiply the RMS value by $\sqrt{2}$. In signal processing, this is generally used to convert RMS values of a fluctuating signal to its amplitude.

Question C.3f Convert the RMS values to the diameter of the protein. Do the new values match the contours of the protein?

D. Test

Please note the following:

- You will do section D without the help of the assistants. Make sure that you understand the contents of section C. Now is the time to ask for assistance if you need further explanations.
- Both the exercises and the test (sections C and D) fall under general university rules concerning plagiarism and exams. Make sure that you play fair and that each of you understands all the content that you submit.
- When you are finished, save the **CoCalc** document as a pdf. Note that there are different options to do so in **CoCalc**. Check that you used a working method. We will only grade pdf documents.
- You need to submit both your answers to section C and section D in a single document. Section D will only be graded if you also submit section C.
- Make sure that both your names and both your student-id's are in the first cell of the document. Documents without names and student-id's will not be graded.

1 Protein folding

- Question D.1a** What is the function of the protein with pdb id 5FJ8?
- Question D.1b** Read the coordinates of this protein into a `dataframe`. Print a list of all the unique residues in the `dataframe`.
- Question D.1c** Some of these residues are not amino acids. What are they? (ignore those labeled 'UNK')
- Question D.1d** Discard all atoms, except the central carbon in each amino acid. Check the result by printing the first 15 rows of the `dataframe`.
- Question D.1e** Write code that counts how many proteins ('chains') are in the complex and print that number.
- Question D.1f** You can use slicing to reduce the contents of your `dataframe`. Use this to discard all columns, except 'resname', 'x', 'y' and 'z' and print the result.
- Question D.1g** Use `df['r']=...` to add a new column to the `dataframe` that contains the distance of each amino acid from the center of mass and plot the probability distribution.
- Question D.1h** One would expect the probability of finding amino acids to be roughly the same within the protein (i.e. equal density everywhere). The plot you just generated does not have a 'flat' region within the protein but goes gradually to 0 for small distances from the center. Explain why this representation is biased. In other words: why is the probability distribution of finding amino acids as a function of r peaked?
- Question D.1i** Calculate and plot a probability density distribution for r that corrects for this.
- Question D.1j** Look up the amino acids that are classified as 'hydrophobic'. Make a list of those in the 3-letter format that is also used in the `dataframe`.

Question D.1k

Plot the probability density distribution for all amino acids, only the hydrophobic amino acids, and the remaining (hydrophilic) amino acids in a single figure with differently colored lines. Are they distributed equally? Would you expect them to be distributed randomly? Why (not)?

Chapter 2

Equipartition theorem

A. Background

In the previous chapter, you have learned that living matter consists of micron-sized cells. Inside the cells, it is crowded with a large variety of molecules. The largest are DNA molecules, being two nanometers thick and up to several centimeters long. The DNA consists of a string of base pairs, which encode the sequence of proteins. Proteins have a large variety of shapes that are defined by the chemical nature of the amino acids that form the protein. The physical link between proteins and DNA is RNA, which is a copy of the DNA. In some ingenious way, this is sufficient to (re-)produce the shape and functions of cells. Proteins achieve this seemingly impossible task by providing structure to the cell and by catalyzing chemical reactions that produce other molecules and consume others. But how do cells achieve a robust and well-organized distribution of molecules?

Before addressing this very complicated question, let's first take a step back and see how molecules in general behave. For this, we need to define the conditions that are relevant for living matter. Since water is the main constituent of the cell we will need to focus on its properties. One of them is that it freezes at 0 °C and evaporates at 100 °C, which already sets a range. But from personal experience, we know that we prefer 37 °C. A few degrees of fever seem to have an enormous impact on our well-being. So temperature is an important parameter for life and we need to look closer into it. We'll therefore start with a simple question (which founded an exciting new branch of physics: statistical physics):

How do the simplest molecules behave when confined in a small space at room temperature?

B. Theory

In this section you should:

- acquire/update the use of conservation of momentum to compute the course of colliding balls
- review knowledge of the ideal gas law
- appreciate how temperature is defined at the molecular scale

1 Billiard balls

To get a grip on the physics of the molecules inside the cell we will start with a very simple model and treat each molecule as a small sphere. The motion of one, or more, of such spheres can be described by Newton's laws using its mass m and radius r . Here we will review the mechanics of colliding spheres. You may think of these spheres as billiard balls. Like in billiards, there are no interactions between the balls or with their environment, except for collisions between balls and between a ball and the wall. Importantly, and this is usually not quite realistic, there is no friction so each ball will keep on rolling until it hits a wall or another ball.

We will discriminate between the speed and the velocity of the balls.

Question B.1a What is the difference between speed and velocity?

In high school, you've learned to solve problems involving colliding objects using the conservation of energy, in this case kinetic energy. When a ball hits the wall we can assume an elastic collision.

Question B.1b Explain the concept of an elastic collision in one sentence.

In this case, it is trivial to calculate the speed of the ball after the collision.

Question B.1c What can you say about the direction of the ball after a collision at an arbitrary angle?

Question B.1d Draw the vectors of the velocity before and after the collision. Decompose both into components parallel and perpendicular to the wall.

For collisions between two balls, there are two variables that we need to calculate: the velocities of both balls after collision. Next to the conservation of energy we therefore need a second equation to obtain the two unknowns. A second conservation law can be deduced from Newton's third law.

Question B.1e State Newton's third law in one sentence. Also, write it down in a formula.

Using the same symmetry argument as in the answer to this question, the time during which ball 1 contacts ball 2 is equal to the time that ball 2 contacts ball 1. This leads to the following equation:

$$F_1 \Delta t = -F_2 \Delta t \quad (2.1)$$

The quantity $F \Delta t$ is called the impulse.

Question B.1f Use Newton's second law to express the impulse in terms of velocity.

The momentum (not to be confused with the moment, as defined by equations 1.2 and 1.3) of the ball is the product of its mass and its velocity and is assigned the parameter p . Like energy, it is a conserved quantity.

Question B.1g Is momentum a scalar or a vector? Why?

Question B.1h Write out the change of momentum of a ball in a formula that contains its velocity before collision $\vec{v}$ and after collision $\vec{u}$.

To calculate the velocities of both balls after collision, we can combine the two conservation laws:

$$m_1 \vec{v}_1 + m_2 \vec{v}_2 = m_1 \vec{u}_1 + m_2 \vec{u}_2 \quad (2.2)$$

and

$$\frac{1}{2} m_1 \vec{v}_1^2 + \frac{1}{2} m_2 \vec{v}_2^2 = \frac{1}{2} m_1 \vec{u}_1^2 + \frac{1}{2} m_2 \vec{u}_2^2 \quad (2.3)$$

Solving the two unknowns, $\vec{u}_1$ and $\vec{u}_2$, from these two equations is quite tedious and is beyond the scope of this course. For $m_1 = m_2$ it is much simpler, and there are two quite trivial solutions.

Question B.1i Calculate the two solutions for $\vec{u}_1$ and $\vec{u}_2$ in this case.

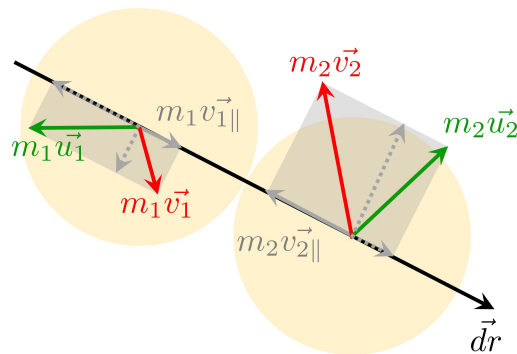


Figure 2.1: Conservation of momentum of two colliding balls.

The situation when two balls collide head-on is depicted in figure 2.1.

Question B.1j Make a drawing (to scale) that shows that momentum, as drawn in figure 2.1 is conserved.

Question B.1k Write down an expression for impulse component $m_1 \vec{v}_1_{\parallel}$ in terms of impulse $m_1 \vec{v}_1$ and a vector connecting the centers of the two balls, i.e. $\vec{dr} = \vec{r}_2 - \vec{r}_1$, where $\vec{r}_1$ and $\vec{r}_2$ are the positions of the balls.
(Hint: Make use of the dot product, rather than explicitly

writing out the angles between the vectors.)
(Tip: It's convenient to normalize $\vec{dr}$.)

Question B.1l Write out the $\vec{u}_1$ and $\vec{u}_2$ in terms of $m_1, m_2, \vec{r}_1, \vec{r}_2, \vec{v}_1$ and $\vec{v}_2$.

Question B.1m If $\vec{r}_1 = [-2, 0]$, $\vec{v}_1 = [2, 1]$ and ball two is at rest at $[2, 0]$, what are the velocities of the two balls after collision?

In simulations we will take discrete time steps to compute the locations of two balls that move to each other, as displayed in figure 2.2a. However, after a time step δt , the balls may have moved so far, that they would overlap (2.2b). Of course, this cannot happen, and we will need to evaluate the positions and velocities at the moment of collision. It is convenient to use a reference frame relative to ball 1. In that case, ball 2 is displaced from ball 1 by $d\vec{r}$ and appears to move with a velocity $d\vec{v}$ relative to ball 1 (2.2c).

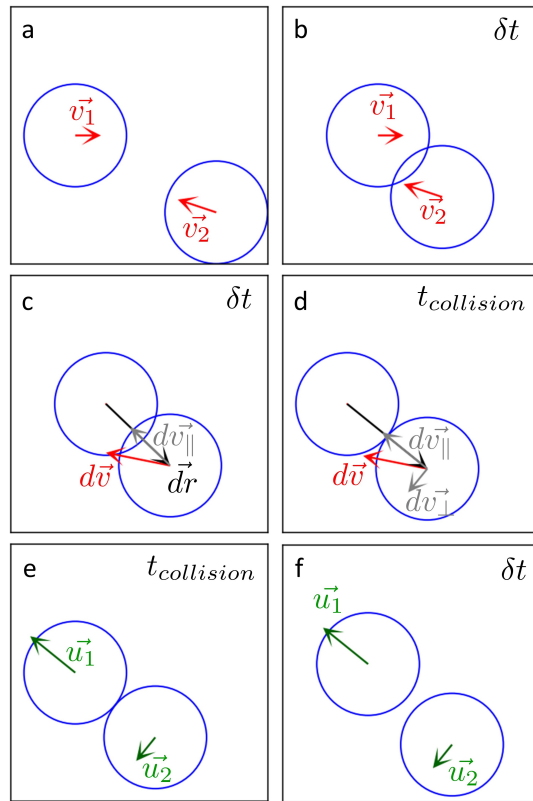


Figure 2.2: Simulating two colliding balls in discrete time steps.

Question B.1n Express the time that the balls overlap, t_{after} , in terms of radius R , relative velocity $d\vec{v}$ and displacement $d\vec{r}$. (Hint: use the dot product to calculate $d\vec{v}_{\parallel}$.)

We subsequently compute the positions of the balls at $t_{collision} (= \delta t - t_{after})$, rather than at δt (figure 2.2d).

Question B.1o Explain in a few sentences what happens next in figures 2.2e and f. In particular, explain how $\vec{u}_1$ and $\vec{u}_2$ relate to $d\vec{v}$ and how the positions of the balls, r_1 and r_2 at time δt can be calculated.

2 An ideal gas

When scaling down to molecular dimensions, the discussion above seems to be much more appropriate to the behavior of ideal gasses than that of the molecules in a cell. In the next chapter, we will see that many of the emergent phenomena in cells stem from the behavior of large numbers of molecules, quite similar to that of ideal gasses, though we will need to take the limitations of our model into account. For now, we will explore what we can learn about such ideal gasses using the model of colliding balls.

Question B.2a What are the assumptions for which a gas behaves 'ideal'?

Question B.2b Write down the ideal gas law; make sure that the number of molecules N is in it.

Note that the unit of energy that we used in Chapter 1, is also present in this equation.

Let's consider a cubic box in which a small number of molecules are placed. The molecules will have a finite velocity when the temperature is not equal to zero.

Question B.2c Given a velocity of v_x and a box size of L , how much time will it take the molecule to move from the left wall to the right wall?

Question B.2d What is the change of momentum of the molecule when it bounces off the wall?

Since momentum is conserved, this also means that the wall experiences the same change of momentum, but in the opposite direction. The force is equal to the rate at which the momentum changes.

Question B.2e Express the force on the wall exerted by N molecules as the change of momentum divided by the time between collisions

Like in the ideal gas law, we are more interested in the pressure on the wall than the force on it.

Question B.2f Modify the above equation, such that the left-hand side only contains the pressure of the wall.

Question B.2g Combine the above expression for the pressure with the ideal gas law and make sure that the dimensions of the box fall out of the equation.

If you do the above correctly, you will end up with a very simple equation that relates the kinetic energy of the molecules to the temperature inside the box:

$$\frac{1}{2}mv_x^2 = \frac{1}{2}k_bT \quad (2.4)$$

This is a remarkable feat known as the equipartition theorem. It applies to all degrees of freedom. So in 3 dimensions one gets the total kinetic energy of a molecule:

$$\sum_{i=x,y,z} \frac{1}{2}mv_i^2 = \frac{3}{2}k_bT \quad (2.5)$$

Knowing the temperature of the gas, which is easy to measure, we can directly calculate the velocity of the molecules. Not all molecules have the same velocity though: in our simulations we will look at the distribution of velocities. Likewise, one cannot define the temperature of a single molecule. The velocity in equations 2.4 and 2.5 therefore represents the average of the population of molecules.

Question B.2h Should one take the average of the velocity squared, $\langle v \rangle^2$, or the average of the squared velocity $\langle v^2 \rangle$? Why?

Question B.2i Look up the mass of an oxygen molecule and calculate its average velocity at room temperature.

Not all molecules have the same speed though. Whenever two molecules collide, they will exchange momentum and the speed of each molecule depends on all the collisions it experienced before. So the velocity will be distributed among the molecules. This is a stochastic process, and we will see many of these processes in this course. Though it may look like a dead end because it makes the calculation of the speed of a molecule an impossibly tedious task, it will turn out to be a blessing in disguise.

C. Exercises

In this section you will:

- set up a molecular dynamics simulation from scratch
- learn how you can speed up computations by avoiding for loops
- get a grasp of the emergent properties of a population of particles

1 Box it up!

To set up a molecular dynamics simulation we start by defining a box and putting several particles in it. Each of the particles is defined by its location and velocity. Next, we will propagate the motion of each particle in small time steps. After each time step, we evaluate the new positions and adjust the velocity of each particle when it collides with the wall or another ball. In this manner, we keep track of all particles and use these records to evaluate their population properties.

We will implement the algorithm in small steps and at the end we will combine the pieces of code into a full molecular dynamics simulation. For each step, we will evaluate if it works properly. This makes debugging much easier. Visual representation of (intermediate) results is key to efficient debugging.

For a useful simulation, we need to make very small time steps and a large number of iterations. It is therefore important to write efficient code: a more efficient implementation of the same algorithm will get you better results.

Let's start and evaluate `n_particles` in a 2-dimensional box of size 1. The positions of the particles will be stored in a `numpy` array `position` and their velocities in a `numpy` array `velocity`. The particles have a size relative to the box size, defined by the parameter `radius`, and will be placed in the box at random positions. The velocities have random directions and initially, all particles have the same speed `v_start`.

Question C.1a Write a function `initialize()` that takes `n_particles = 10`, `v_start` and `radius` as inputs and returns two `numpy` arrays `position` and `velocity`. Make sure that the particles don't cross the borders of the box. Print the result for a set of reasonably chosen input parameters. (Hint: use the `numpy` function `random.uniform()`).

For a better evaluation of the result, we want to plot the particles and the box to scale. We will use `Matplotlib.pyplot` for it (make sure that you imported it as `plt` in this cell or a cell above!). You will get a nice layout when you use the code in listing 3 for it.

Question C.1b Copy this code and plot the initial distribution of particles in the box. (Hint: make sure that you use `plt.show()` for a nice result)

Question C.1c Create a new function `advance(position, velocity, dt)` to adjust the positions of the particles to `new_position`. Choose `dt` such that in combination with velocity, you get a small, but visible displacement. Plot the result.

You can use the newly defined function `advance()` iteratively to move the particles through the entire box. Though we generally like to avoid `for` loops for efficient code, it is not possible in this case. This is because the input of the next iteration requires the output of a previous iteration. So this calculation can not be done in parallel and must proceed sequentially.

```
def plot_box(position, radius, color='blue'):
    plt.close() # close any previous plots
    fig, ax = plt.subplots(figsize=(3, 3))
    ax.set_aspect('equal', 'box')
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    plt.xticks([])
    plt.yticks([])
    circles = []
    for p in position:
        circle = plt.Circle(p, radius, fill=False, color=color)
        ax.add_artist(circle)
        circles.append(circle)
    return fig, circles
```

Listing 3: Plot the position of particles in a box using circles.

Question C.1d Compute new `position` arrays in 50 steps. Store the result in a list named `positions` using the method `.append()`. In the end, you can use the `numpy` function `asarray()` to convert the list `positions` to a `numpy` array. Print the size of this array.
(Tip: make sure that you create an empty list `positions` before you use the `.append()` method)

If everything goes well, your `positions` array has 3 dimensions and has a shape of `[50, 10, 2]`.

Question C.1e Explain what each of the dimensions represents.

It will be nice to evaluate the result of our simulation as a small movie. In listing 4 is the function `animate_box()` that can be used, in combination with `plot_box()` to create an animation.

Question C.1f Look at the structure that creates a list of `titles`. Such a structure is called 'list comprehension' and it speeds up the code significantly. It also makes the code more concise and thus easier to understand and maintain. Write some code that results in the same list `titles` but using a for loop instead and compare the code.

Further details of this code can be found in the `Matplotlib` documentation but are beyond the scope of this course. It is sufficient to just copy and paste it into your notebook.

Question C.1g Copy the code of listing 4 into a new cell and test it by calling the function `animate_box()` with appropriate parameters. What happens with the particles after many steps?

```

from matplotlib import animation
from IPython.display import HTML

def animate_box(positions, radius, frames=None):
    # Put the call to this function at the end of the CoCalc cell

    def add_frame(i, positions, titles):
        plt.cla()
        plt.xticks([])
        plt.yticks([])
        ax = plt.gca()
        for p, c in zip(positions[i], circles):
            c.center = p
            ax.add_artist(c)
        plt.title(titles[i], fontsize=12, loc='left')
        return circles

    fig, circles = plot_box(positions[0], radius, color='blue')

    if frames is None:
        frames = range(len(positions))
    titles = [f'frame {frame}' for frame in frames]

    anim = animation.FuncAnimation(fig, add_frame, range(len(frames)),
                                   fargs=[positions, titles])

    return HTML(anim.to_html5_video())

```

Listing 4: Code to make a movie of the positions of circular particles.

(Tip 1: It may take a while for this function to finish.)

(Tip 2: If your box is empty, you may need to run the function `initialize()` again. Why?)

There are a couple of practical issues with the code of `animate_box()` in combination with CoCalc that you may experience:

- The movie is only displayed properly if you put it at the very end of the cell.
- It also plots the last frame below the movie, which is convenient when you make a pdf file.
- The number of frames that can be displayed is limited, so if you experience an error or a long lag, you may want to reduce the number of frames.

An often-used method to keep the number of particles in the box constant is to use 'periodic boundary conditions': whenever a particle crosses the walls of the box, it returns on the opposite side with the same velocity. For this, we need to evaluate the position of each particle after each step and correct it when necessary. This simulates the situation where you have a large volume and a high concentration of particles and it is sufficient to keep the concentration of particles constant.

Question C.1h Modify the function `advance()` such that you pass a parameter that sets the boundary condition. The interface of the function will be: `advance(position, velocity, dt, boundary=None)`. Then modify the body of the function and test the result by doing a new simulation with the parameter `boundary='periodic'`. Show the movie.

However, periodic boundary conditions are not what we want: we want the particles to bounce off the walls. For this, you will need 'reflecting boundary conditions'. As you have seen in section B, you need to adjust the velocity, rather than the position. This means that the function `advance()` not only needs to return a `new_position` array but also a `new_velocity` array, that can be iterated through the simulation.

Question C.1i Adjust the function `advance()` once more so it can handle reflecting boundary conditions.
(Tip: first make a Boolean array for each position that tests whether the particle is outside the box, then use slicing to adjust the velocity of those selected particles. In this way, you don't need to use a `for-loop`.)

Do you observe particles that just cross to boundary before reflecting? There may be 2 causes: i) you may not have taken the size of the particle into account, but rather the center of it. ii) Your time step is too big and though the velocity is adjusted when the particle crosses the wall, its position is not.

Question C.1j Check your code and adjust it so the particles cannot enter the area that is closer than one radius away from the wall.

The second flaw can be addressed by taking smaller time steps. However, then you will end up with shorter trajectories, or you need to increase the number of iterations proportionately.

2 When two particles collide...

In the previous set of questions, you generated the code to simulate the trajectory of several particles, by taking small steps and, given their velocity, updating their position. You have also implemented the presence of a barrier and learned different boundary conditions, taking the physical size of the particles into account. Finally, you were provided with the code to display the particles in the box in a snapshot or as a movie. This was critical to evaluate the outcome of the simulation.

However, the particles still display 'non-physical' behavior: they don't notice each other, and they can even overlap. You could simulate the trajectories of each particle separately, combine the trajectories, and not notice the difference.

Here we will look at the collisions of the particles themselves, using the law of conservation of momentum. This will be a tedious task, and it will become important to write efficient code, so we can still make a large number of iterations without very long waiting times. The reward is that we will be able to explore new physics: there will be emerging phenomena that can not be captured by analyzing single trajectories independently.

Let's first address the problem of overlapping particles.

Question C.2a Use the previously coded function `initialize()` to generate a set of 4 particles, of which some overlap. To increase the odds, use a relatively large radius. Plot the outcome and print the coordinates of the particles.

Question C.2b Create a function `get_collisions(position, radius)` that outputs a list of pairs of particles that overlap. For the time being, you may use a `for`-loop. Print the result.

When initializing, we don't want any overlapping particles. For this, we need to modify the initialization of the positions. It is also good at this moment to realize that the box may be too small for the number of particles that you want. A practical and robust solution to always get an outcome is to try adding a particle, test it for overlap with existing particles, and discard it when there is overlap. Iterate this until you have enough particles, or if you have tried too many times (say 2 times the number of particles).

Question C.2c Modify the function `initialize()` according to the above algorithm and test the outcome by plotting a crowded box. Make sure that the shape of the array `velocity` cannot be different than that of `position`.

Now we need to implement collisions of the particles. In section B, you derived the equations for the velocities of two colliding balls and expressed these as a function of the initial velocities and positions, as depicted in figure 2.1. You can conveniently use the `numpy` function `dot()` for this. As you did in the homework, we assume that all particles have the same mass. In doing so, all the terms m disappear from the equations.

Question C.2d Convert the theory from section B to a function `handle_collision(pair, position, velocity)` in which the input parameter `pair` represents the indices of the two colliding particles as generated by the function `get_collision()`. The `numpy` arrays `position` and `velocity` contain the locations and velocities of the particles, as generated by the function `initialize()`. Assume that the particles have the same mass. The function should return an array `new_velocity`. Test the function by manually generating the arrays `position` and `velocity` to correspond with $\vec{r}_1 = [-2, 0]$, $\vec{v}_1 = [2, 1]$ and ball two is at rest at $[2, 0]$. Compare with your theoretical answer.

Question C.2e Next, change the function `advance()` to include collisions. Make sure that you modify the latest version that has reflecting boundary conditions. Test the code by running a

small simulation with a small number of particles of sufficiently large size and iterate it long enough to see several collisions.

If you look carefully, you may see particles that appear to stick to each other. This happens when the positions of the particles after collision overlap so much, that in the next time step, they do not move far enough away from each other, so they still appear to collide. When this happens, their velocities are reversed again, and these small alternating forward and backward moves proceed until the next collision may break it loose. In section B, you worked through the steps to correct for such nonphysical overlaps.

Question C.2f Implement this correction in the function `handle_collision()`. Note that this function will now need the `old_position` array, as well as the time step `dt` as inputs, and you will create not only a `new_velocity` array, but also a `new_position` array. (Tip: Don't worry too much if this is beyond your Python powers. If that's the case: skip this question, but carefully review the answer model and copy the code from there. It is expected that you understand what's going on.)

Now is a good time to wrap up: you have created several functions that together make a full molecular dynamics simulation. The functions are in different cells of the **Jupyter Notebook**. This may confuse **Jupyter** when you don't run cells in the right order. The solution is to collect all functions in a single cell.

Question C.2g Copy and paste all relevant functions in a single cell and write some code at the bottom that demonstrates that the entire simulation can run in it. To test this behavior select **Kernel>Restart Kernel** from the **CoCalc** menu. Click on the cell that includes the entire simulation to make it active and press **CTRL-Enter**. (Tip: you will want to include all imports as well.)

3 Hit the gas!

The molecular dynamics simulation of particles in a box is a paradigm for many phenomena in statistical physics and gives insight into the fundamental processes that are often hidden by the small size of the particles or by the large number of them. Before we continue, however, we want to speed up the code, to allow for longer simulations, larger simulations, or smaller time steps.

To increase the speed in **Python**, the most important step is to remove **for**-loops, whenever possible, and to replace these with a vectorized algorithm. Many problems that require calculations with large data arrays allow for vectorization: rather than dealing with one element at a time, you can do the calculation on an entire array. **Numpy** is particularly good at vectorizing calculations. Moreover, behind the scenes **Numpy** uses compiled **C++** code, which is much faster than generic **Python**. So use **Numpy** arrays whenever you can.

List comprehension is another way to optimize code. Though there is still a **for**

statement within the brackets, **Python** knows that it can run the iterations in parallel and this yields a significant speed increase.

In addition there are specialized packages that can optimize the execution by using faster interpreters, GPU, or multiple CPU cores. We will not make use of these.

If **CoCalc** gets stuck, the odds are that your implementation of the code is sub-optimal, or that you have chosen parameters that are too demanding in terms of memory. Here, we will implement a couple of optimizations to our code to allow for better performance. But maybe more importantly, we will add a timer so you can monitor the progress of the code and identify time-consuming steps that benefit most from code optimization.

We will use the progress bar from the module **tqdm**.

Question C.3a Look up the documentation of **tqdm**. Import the function **tqdm()** from this module and integrate it into the outer loop of your simulation. How many steps per second can you simulate for 10 particles?
(Remember: you don't need to copy all the code from the previous cell, if you have run it before. Just call the functions that you need for the simulation.)

You can also add a progress bar in the function **animate_box()**. For this, add the code in listing 5 to the last line in **animate_box()**. Note that the function **tqdm()** accepts the parameter **desc** to add a small descriptive title.

Question C.3b Update the function **animate_box** to include the progress bar and also add a description 'Simulation' to the main loop. Test the result. Which part of the code takes the longest?

```
for _ in tqdm(frames, desc = 'Animation'):  
    anim._step()
```

Listing 5: Add a **tqdm** progress bar to the function **animate_box**.

If everything went well, you see that the actual calculations don't take a lot of time, but the visualization, i.e. the function **animate_box()** is the culprit. The function largely uses standard functions of **matplotlib.pyplot**, which should be reasonably optimized. It is however well-known that graphical display is computationally intensive. Displaying the movie after it was created by **animate_box()** also took significant time, but we will accept this as is. The best we can do is to limit its use. This may sound disagreeable, but accurate simulations require small steps, and it may not be necessary to visualize every step.

Likewise, it may not be necessary to save the positions for every iteration. Instead, let's create a list of frame numbers named **frames**, and only save the outcome of those frames.

Question C.3c Create a list of `frames` containing `n_frames` integers that are distributed over `n_iterations`, start with 0 and end with `n_iterations-1`. Use a single line of code.

Now we can increase the size of our simulation.

Question C.3d Keep the number of iterations equal to 100, but increase the number of particles such that the simulation takes less than about 30 seconds. Decrease the radius when necessary. How many particles can you simulate within a reasonable time?

You must have noticed the long lag before the simulation started. This must be the function `initialize()`.

Question C.3e Add a `tqdm` progress bar to the time-consuming loop within this function. Add the description 'Initialization'. Is this function indeed responsible for the long lag?

Still, there is a lot of performance to gain. When you wrote the function `get_collisions()`, you probably used two nested `for` loops. Though this is conceptually a very simple algorithm, it is very time-consuming. And because each iteration of the functions `initialize()` and `advance()` call this function, a more efficient algorithm would have a large impact on the speed of the entire simulation.

It turns out that finding clusters of closely spaced points in n -dimensional space is a problem that is often encountered in modern data science. We can benefit from the optimized code that has been developed for it.

```
from scipy.spatial import cKDTree

def get_collisions(position, radius):
    return cKDTree(position).query_pairs(np.max(radius) * 2)
```

Listing 6: Using `kdtree` to find colliding particles.

Question C.3f Replace the function `get_collisions()` by the code in listing 6 and test whether there is a speed increase.

If you did not get a significant speed increase, it may be that `CoCalc` still had the function from a previous cell in memory. Better to start with a fresh cell again.

Question C.3g Gather the modifications to the simulation in this section into a new single cell again. You will use this code in the test and in the next chapters.

D. Test

Please remember:

- Make sure that both your names and student id's are on every document that you submit.
- Only submit pdf's and check that CoCalc produced the correct pdf.
- Make a new cell for each question and start each cell with printing the question number.
- Adjust the layout of your plot, such that it optimally displays the data.

1 Maxwell-Boltzmann

Question D.1a There may still be nonphysical incidents in your simulations (particles that penetrate the wall, overlap, ...). Explain how these can happen, and which parameter in your simulation needs to be adjusted to minimize these.

Question D.1b Run a simulation with at least 1000 particles, at least 1000 iterations, and a speed of 1. Make the radius of the particles such that they occupy 25% of the available area. Plot the probability density distribution of the x-coordinate and the y-coordinate of all particles in all frames. Is the entire box visited by the particles?
(Tip: There is no need to show the movie. To save time, you can just plot a single frame if you need to check the distribution of particles.

Question D.1c Plot the probability density distribution of the velocity of the last frame for 3 simulations with `v_start = 0.5, 1, and 2`. Why is it centered at zero? What happens with the width of the distribution when you double the start velocity?

Question D.1d Adjust the code, such that it also keeps track of the velocities. Run a new simulation with the same parameters but with 10000 iterations and a time step of `1e-5`. Plot the probability density distribution of the velocities in five equally spaced points in the iteration. Label the plots with the total number of iterations of the simulation. Explain in terms of momentum what makes the distribution change.

Question D.1e Compute the average speed of the particles in each frame in a vectorized manner and plot it as a function of the number of steps. Make sure that your plot has at least 100 points. What is unexpected in this plot and what physics assumption does this violate? We will ignore this shortcoming and proceed with the part of our simulation that does not appear flawed.
(Tip: Make use `np.sum()` and the parameter `axis=...`)

Question D.1f Plot the histograms of the probability density distribution of the speed for 5 equally distributed time points. After how many iterations did the simulation reach a steady state?

Question D.1g To get better statistics, change the code to keep track of the last 2500 iterations. Plot the probability density distribution of this entire population. What is the mean speed? Is the speed distributed symmetrically around the mean?

The distribution that you just found was derived theoretically by James Maxwell in 1860 and further studied by Ludwig Boltzmann 10 years later and is referred to as the Maxwell-Boltzmann distribution. In 2 dimensions the probability density distribution follows:

$$P(v) = \frac{m}{k_B T} v e^{-\frac{m v^2}{k_B T}} \quad (2.6)$$

Question D.1h You actually don't need the mass or temperature to calculate the result if you use the equipartition theorem. Plot the probability density distribution that you found in the previous question (use a scatter plot, rather than a line plot) and plot equation 2.6 using the just obtained mean speed in the same plot.
(Tip: to get a nicer curve you may want to calculate the equation 2.6 with more points than the number of bins of the histogram)

The Maxwell-Boltzmann distribution for 3 dimensions is:

$$P(v) \propto v^2 e^{-\frac{m v^2}{k_B T}} \quad (2.7)$$

You don't even need to know the proportionality factor to plot the full probability density distribution of the velocity.

Question D.1i Plot the Maxwell-Boltzmann distribution for the molecules O_2 and N_2 and CO_2 in air at room temperature. Which molecules moves fastest?

Chapter 3

Life at low Reynolds numbers

A. Background

In chapter 2 you studied the behavior of an ideal gas at the molecular scale. At finite temperatures, the equipartition theorem (eq. 2.5) tells you that all molecules will move with a speed that is proportional to the square root of the temperature. Moreover, though all molecules have different velocities, their speed will be distributed in the same manner. If you wait long enough, the speed probability density will follow the Maxwell-Boltzmann equation (eq. 2.7).

But of course, the contents of a cell can hardly be modeled as an ideal gas: it's a liquid in which the molecules do interact. Moreover, there is a large variation of molecules in terms of size and mass. Here, we will stretch the analogy with an ideal gas a bit further and explore how (differences in) size and mass affect the behavior of individual molecules in an ensemble of confined molecules. Whereas the molecules in the ideal gas appeared to move like macroscopic bouncing balls, it turns out that the movement through water at the microscopic scale can be counter-intuitive. In the world of small molecules, frictional forces dominate. But how does that work at the molecular scale?

How do small objects, like proteins and bacteria, move through a crowded watery environment?

B. Theory

In this section you should:

- Familiarize yourself with two regimes of fluid dynamics
- Reacquaint yourself with Stokes' law
- Derive the Stokes-Einstein equation

1 Friction forces

Until now, we have explicitly kept energy losses out of our analysis. But when objects move through a gas or a liquid, we know that friction causes the object to slow down. Friction increases with velocity, and there are two formulae that relate the drag force F_d to the velocity v :

$$F_d = 6\pi\eta Rv \quad (3.1)$$

and

$$F_d = \frac{1}{2}\pi c_d \rho R^2 v^2 \quad (3.2)$$

Equation 3.1 is known as Stokes' law. Equation 3.2 is attributed to Lord Raleigh.

Question B.1a Use both equations to calculate the terminal velocity (the highest velocity that it can reach) of a tennis ball falling from a tall building. Look up or estimate the unknown values.

Of course only one of these equations can be valid, and which one depends on the type of motion of the gas (or fluid) that causes the friction. Generally, Stokes' law is valid when friction is relatively high. In that case, the fluid will flow around the ball in a laminar, ordered manner. The drag force only becomes proportional to the square of the velocity of the fluid when the difference in pressure between the front and back of the object becomes so large that turbulence arises. Which regime applies depends on several factors that combine in the Reynolds number:

$$Re = \frac{\rho L v}{\eta} \quad (3.3)$$

Question B.1b Look up the dimensions of each of the parameters in this equation. What is the dimension of the Reynolds number?

The Reynolds number represents the ratio between inertial forces and frictional forces. When inertia dominates, Re is much larger than 4000, and equation 3.2 should be used. When Re is smaller than 2000 inertia can be neglected, and Stokes' law applies. In between, the flow of the fluid will be somewhere between laminar and turbulent and the physics is extremely difficult to manage.

Question B.1c Calculate the Reynolds number for the falling tennis ball. Which regime applies?
(Hint: it does not matter which of the outcomes of the first question you use)

The transition between both regimes is gradual, but the range of Re is large, see figure 3.1. It is usually sufficient to have an order of magnitude estimate of it to know which regime applies. The precise definition of the length L doesn't

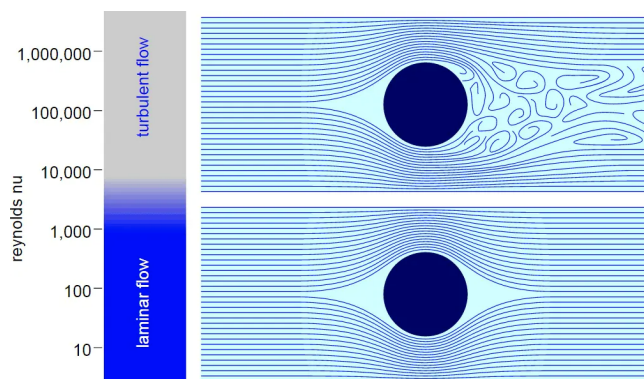


Figure 3.1: The transition from laminar flow to turbulent flow is characterized by the Reynolds number. (From <https://www.nuclear-power.com/>)

matter for such an estimate: the typical length scale of the object that moves through the flow, or rather the dimension of the perturbation of the fluid, is good enough.

Question B.1d What Reynolds number applies for a bacterium moving through the water at a speed of $30 \text{ } \mu\text{m/s}$?

To see the consequences of friction at a small scale in a different manner, calculate the time constant with which the velocity decays when we don't neglect inertia. In that case, one can equate the friction force to the inertial force.

Question B.1e Express the acceleration as $\frac{dv}{dt}$ and solve the differential equation for v . What is the decay time?

Question B.1f Fill out typical numbers for the cruising bacterium. How long does it take after stopping with swimming before it's velocity has reduced to $1/e$? (Tip: use a density of $1.3 \cdot 10^3 \text{ kg/m}^3$ for the bacterium)

Question B.1g How far does the bacterium move when it stops swimming?

If an entire bacterium falls into the regime of low Re numbers, then that is definitely the case for all objects within a cell: the dimensions are smaller and the viscosity of the fluid in the cell can be expected to be higher than that of pure water.

2 Gravity

If inertia doesn't play a role, would gravity be important for the processes of life at the molecular scale? It is hard to imagine life without gravity. But it may be misleading to make an unsubstantiated guess at the answer. Let's first go back to the movement of the molecules in an ideal gas.

Question B.2a Calculate how high a molecule of oxygen could come if all its kinetic energy would be converted to potential energy. (Tip: assume that temperature is constant)

Of course, the molecule will collide with other molecules on its way up, so the above calculation is only a very rough estimate. A better way to quantify the distribution of molecules due to gravity is to invoke Boltzmann's law: in equilibrium, so when there is a stable *quasi-static* distribution of particles, the probability of finding a particle with energy E_i is:

$$P(E_i) = \frac{e^{-E_i/k_B T}}{Z} \quad (3.4)$$

Z is simply a normalization constant that ensures that the total probability adds up to 1:

$$Z = \sum_i e^{-E_i/k_B T} \quad (3.5)$$

Z is also known as the partition function. The term *quasi-static* implies that this is not a static situation in the sense that nothing moves. On the contrary: there is a continuous movement of all particles, and all particles can participate in all states, just like you observed in the molecular dynamics simulation of the ideal gas. Note that the Maxwell-Boltzmann distribution (eq. 2.6) is a combination of the equipartition theorem (eq. 2.4) and the Boltzmann equation.

Question B.2b Using the potential energy of an oxygen molecule, what is the decay-length of the oxygen concentration? How does it compare to the thickness of the atmosphere?

Having two different species, the situation is a bit different: when a particle moves up or down not only its own potential energy changes, but also the potential energy of the other particles that will be displaced. So when a particle sinks in water its potential energy goes down, but an equal volume of water is displaced and gains potential energy. Or, when the density of the particle is less than that of water, it will lower its potential energy by floating.

Even a small difference in density will yield a large difference in potential energy for large, macroscopic particles, and therefore they will float or sink. For smaller particles, the situation can be more interesting, and they can be dispersed in the fluid. Raw milk, for example, is a dispersion of small fatty particles ($\rho = 910 \text{ kg/m}^3$) in water, see Fig. 3.2.

Question B.2c What is the decay length of the distribution of fat droplets in milk?
(Tip: calculate the effective mass of the droplet in water first.)

Your result does not directly relate to the experience that you have: a carton of milk appears to be a homogeneous solution, rather than a pack of water with a fat layer on top of it. However, the first step of making butter from raw milk is to wait until the fat separates from the water. It appears that milk is not in equilibrium, so eq. 3.4 cannot be used. To keep the milk from spilling, consumer milk is first homogenized into smaller fat droplets (among other processes), before its bottled.

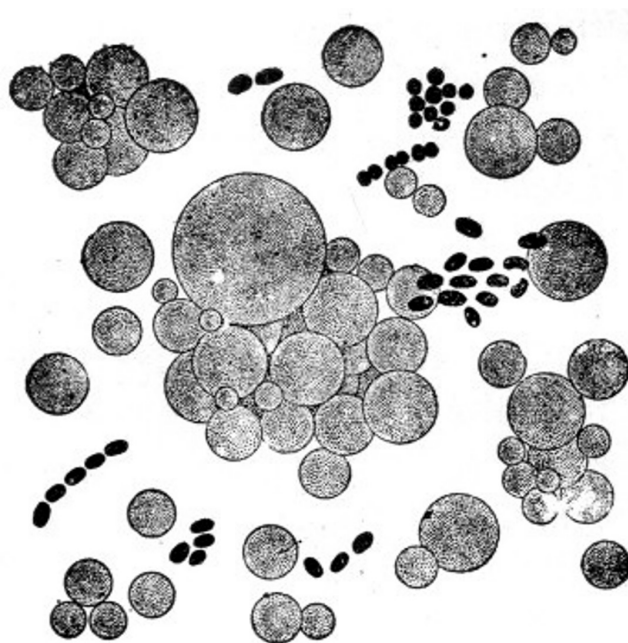


Figure 3.2: Microscopic appearance of raw milk, showing relatively large fat globules. Scale reference: the smaller dark and oval-shaped structures are bacteria.

Question B.2d How long does it take a fat droplet to move from the bottom to the top of a milk carton? And how small should the droplet be to extend that period to 2 weeks?

Question B.2e Argue if sedimentation (or buoyancy) is relevant inside cells.

3 Stokes-Einstein

Though gravity may not be very relevant at the molecular scale, friction is. In this question, we will look closer into the effects of friction at the molecular scale. This may seem a bit odd at first sight, as Stokes' law has viscosity as one of the parameters, and viscosity is a bulk property. However, making the connection between the observable bulk properties of a medium and the mobility of its molecules was the first major scientific contribution of Einstein (Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen, 1905, *Annalen der Physik*). The impact of this paper was so big because it suggested an experimental validation of the *molecular*-kinetic theory of heat at a time when the concept of molecules was controversial. The paper also provided the best way, at that time, of estimating Avogadro's number. The corrected value from Einstein's paper was accurate to one significant digit. One could even argue that this proof of the existence of molecules and atoms was a first step toward quantum mechanics.

Let's start with the molecular dynamics simulation that you set up in the previous chapter. We want to introduce a force term into the position change of

the particles. This force may be gravity or any other constant external force. Since the collisions are instantaneous, the force will only affect the trajectory between collisions, not the collisions themselves.

Question B.3a Take the formula for computing the displacement Δx after time step Δt and add a term that takes into account the acceleration in response to a force F .

Question B.3b Take the time average of both sides of the equation. One term falls out of the equation.
(Hint you have seen this term vanishing before!)

Question B.3c Rearrange the equation, such that the force F is on the left-hand side. You can drop the average sign. Substitute $\frac{\Delta x}{\Delta t}$ with v .

If you did it correctly, you end up with an equation that is very similar to Stokes' law 3.1.

Question B.3d Let's call all the terms on the right-hand side except the velocity, the friction constant ξ . What is ξ in terms of the mass of the particle?

It seems that there is little progress: we can only relate the friction coefficient to a term that contains the arbitrarily chosen time step of the simulation. To follow Einstein's thoughts, we need to bring in a different line of reasoning. We could simplify our simulation even further by following the position of a single particle in time and allowing it to make discrete steps. Each collision displaces the particle by a distance $k\Delta x$. To take care of the random direction of the colliding particle we take the number k arbitrarily from the set $[-1, 1]$.

Question B.3e What is the average value $\langle k\Delta x \rangle$? And what is the average $\langle (k\Delta x)^2 \rangle$?

Now express the position of a particle after N steps, x_N , in terms of the position after $N - 1$ steps:

$$x_N = x_{N-1} + k\Delta x \quad (k = \pm 1) \quad (3.6)$$

Question B.3f Work out the expression for $\langle (x_N)^2 \rangle$ in terms of x_{N-1} and make sure that terms that have an average of 0 are removed.

Question B.3g Propagate this step and express $\langle (x_N)^2 \rangle$ in terms of x_{N-2} .

When you propagate this further, up to the first step, you will end up with a very simple solution:

$$\langle (x_N)^2 \rangle = N\Delta x^2 = \frac{t}{\Delta t} \Delta x^2 \quad (3.7)$$

For the last term, we used $N = t/\Delta t$, in which t is the total time that we follow the particle. Equation 3.7 is a very important result, that we will want to check in the simulations: the mean squared displacement is proportional to time. Lets call the proportionality constant $2D$, resulting in:

$$\langle x^2 \rangle = 2Dt \quad (3.8)$$

As a last step, we can multiply the friction coefficient ξ and the diffusion constant D .

Question B.3h Use the intermediate terms that you derived in the previous two lines of thought and show that the result yields the Stokes-Einstein equation:

$$\xi D = k_B T \quad (3.9)$$

This result is quite special: the friction constant ξ depends on the viscosity of the medium, which is easy to measure, and the size of the particle (eq. 3.1), which is also easy to measure if it is big enough to see. The diffusion coefficient D is also easy to measure if you follow the particle in time. On the right-hand side, there is temperature and Boltzmann's constant. The later connects to the number of molecules via the ideal gas law:

$$PV = Nk_B T \quad (3.10)$$

Thus, one could experimentally verify the molecular-kinetic theory of heat. The trick is to find a particle that is big enough to be observed, but small enough to yield a measurable displacement. And indeed, Jean Perrin experimentally verified Einsteins prediction in 1913.

C. Exercises

In this section you will learn:

- How to code and use an MD simulation that allows for multiple particle sizes
- The implications of size, mass, and force on the outcomes
- The molecular origin of friction and dissipation

1 Onesies are out of fashion

We'll continue with the molecular dynamics simulation but now we want to make it more realistic: not every particle has the same size, and associated with that, not every particle carries the same weight. The code is provided in your CoCalc assignment folder as the file `MDsimulation.py` and contains several changes as compared to the code you wrote last week. We will go through them step by step. The purpose of the series of questions here is to understand the module `MDsimulation.py`, up to the point that you could have written it yourself.

At the very bottom, you can find the function `demo_md()`. This provides the example use of all the functions in the module. You can import the module like any other Python module when the file is in the same directory as the notebook that you are working in.

Question C.1a Import `MDsimulation`, run the function `demo_md()` and confirm that you have a working simulation by looking at the result. What boundary conditions were used?

The first modification as compared to the last chapter is that each particle has its own radius, as defined by the `numpy` array `radii`. This change has many implications. For starters, when you use this array in the function `plot_box()` you'll want circles with different `radii`.

Question C.1b Generate a set of 5 particle positions using `np.random.uniform()` and a corresponding array `radii` with increasing size. Don't use the function `initialize()` for this, just `numpy`. Make sure the sizes are large enough to fill most of the box. Use the function `plot_box()` to plot the particles and confirm that it works as expected.

Probably you will see some overlapping particles. To address this issue, the function `get_collisions()` requires a major adjustment. The previous implementation relied on an algorithm from `kdtree` (see listing 6) for fast detection of overlapping circles. This algorithm can only handle particles with the same radius. In listing 7, you can find an alternative that is slightly less efficient but can deal with various `radii`. To keep the code simple we don't use `Class`-es here. Instead, we use two `globals` (which is not the most elegant solution but it does the job).

Question C.1c Explain in words how using two `globals` helps to speed up the code?

```
from scipy.spatial.distance import pdist

def get_collisions(position, radius, init=False):
    global pairs, min_distance
    if init:
        pairs = np.asarray(np.triu_indices(len(position), k=1)).T
        min_distance = [radius[i] + radius[j] for (i,j) in pairs]
    return pairs[pdist(position) < min_distance]
```

Listing 7: Modification of the function `get_collisions()` to handle particles with different `radii`.

The function `initialize()` also needed major changes. It uses the array `radii` as an input and determines the number of particles based on the size of the array.

Question C.1d Use the function `initialize()` with the previously created array `radii` to generate a new set of positions. Plot the new set of particles and check that none overlap. Why is it useful that the function produces a new array `radii`?

By default, the first particle starts in the center of the box. In addition, the function `initialize()` first sorts the particles according to size (largest size first), before placing them in the box.

Question C.1e Why is it convenient to first sort the particles according to size?

Question C.1f The array `velocity` is calculated differently now. Explain the difference, and why there is a factor $\sqrt{2}$ in it.

The varying sizes of the particles have two consequences for the function `handle_collisions()`: it needs a new function `get_collisions()` to detect overlap and it needs to weigh in the different masses, and therefore different momentum. This is done according to figure 2.2.

Question C.1g Check that the conservation of momentum is implemented properly. Why is `velocity[i]` added to both `velocity[j]` and `velocity[i]` in the last lines of this function?

The function `advance()` only required small changes, but checking the boundaries is now slightly more involved. Boundaries are now checked and acted upon in the new function `handle_wall()`. Each wall is processed independently, allowing you to have mixed options. The type of boundary condition is now set in the parameter `walls`.

Question C.1h What is the default boundary condition when the parameter `wall` is not set? How do you call this function if you want periodic boundary conditions for all walls?

The function `handle_wall()` takes care of adjusting the position of a particle after hitting a wall in a similar fashion as you implemented before when two particles collide, as depicted in 2.2. This should make the simulation more accurate.

It's worthwhile to check whether the new simulation obeys the law of conservation of momentum. For this, you will need to set up a new simulation that has similar functionality as the function `demo_md()` but uses your own parameters. You want to do this in a new cell in your notebook.

Question C.1i Compute the average speed as a function of the number of steps in the simulation. Use at least 250 equally sized particles with a mean speed of 10, that occupy 25% of the area and compute at least 10^4 steps with $\text{dt} = 10^{-5}$. Apply reflecting boundary conditions. Is momentum conserved?

(Tip: It is sufficient to plot the speed, rather than the momentum)

Question C.1j Compare the probability density distribution of the speed with the expected Maxwell-Boltzmann distribution for the used starting speed.
(Hint: You can invoke the equipartition theorem and use the average speed to eliminate the unknown temperature)

2 May the force be with you

In section B you derived the Stokes-Einstein relation by introducing a force on the particles. Here you will do the same. You do not need to edit the file `MDsimulation.py` (Like you don't want to mess with `numpy` or any other module). In stead, you can redefine a function by implementing a new version with the same name in a cell of your notebook. `Python` will use the function that was called last.

Question C.2a Write a new function `advance()` that accepts `force` as a parameter. For simplicity, apply the force only in the vertical direction, like gravity would do. Run a short simulation in which you can observe the effect of `force`. Choose the force and the other parameters such that you can observe a sedimentation effect. Describe what you see.
(Tip: use periodic boundary conditions for the vertical walls.)

Question C.2b Plot the probability density function for the vertical position of the particles of 4 iterations equally spaced in time. Choose bins that offer a fair compromise between the number of bins and the noise per bin. Has the simulation reached equilibrium?

Question C.2c Repeat the simulation, but now accumulate the positions from a large number of iterations after the simulation reaches equilibrium. Make a scatter plot of the probability density distribution for the vertical position. Use a logarithmic scale for the y-axis. Does the profile resemble that of a Boltzmann distribution? How does the log-scale help to assess this?

D. Test

1 Bring in the heavy guys

- Question D.1a** Examine the code `MDsimulation.py`. Explain why/how the first particle always starts in the middle. Also, explain why it makes sense to create an array `mass` that is proportional to the square of the radii.
- Question D.1b** Run a simulation with 199 small particles and 1 larger particle. Good parameters are: `dt = 1e-5`, `n_iterations = 1e4`, `n_frames = 100`, `n_particles = 200`, `start_speed = 1`, `radius = 0.02`. Make the large particle 5 times bigger. Use periodic boundary conditions. Animate the box to observe the result.
- Question D.1c** Plot in one figure the speed of the large particle and the average speed of the small particles as a function of time. Make the y-axis range from 0 to 10. Explain why the speed of the large particle changes.
- Question D.1d** Plot the total momentum of all particles. Make the y-axis start at 0. This simulation cannot handle simultaneous collisions of more than 2 particles. Would this affect this simulation more than when all particles are small?
- Question D.1e** Repeat the simulation but now make the large particle start with a 10 times larger speed than the others. Compare the plot of the speed with the previous simulation. Explain the differences and use the terms friction and dissipation.
- Question D.1f** Change the simulation such that 100 small particles start with a speed of 10 but the large particle starts at rest. Plot the trajectory of the x and y coordinates of the large particle for 1000 equally spaced steps.
- Question D.1g** Repeat the simulations 20 times. Save the x,y trajectories of only the large particle in an array `traces`. Plot all the x and y traces as a function of the number of steps in the same figure with the same color. Also, plot the ensemble average of the x and y positions as a function of time in the same plot (and color it black). Evaluate the result. Do you observe a trend?
(Tip1: No need to plot or animate the box for each simulation, just run it in a for-loop.)
(Tip2: Test your code using small and short simulations before you run it the final time.)

Question D.1h

Plot in the same manner the displacement from the start position and the mean displacement. Would you expect the latter to be linear? Does it follow the expected trend? (Tip: No need to run the simulation again if you stored the array `traces`)

Chapter 4

Diffusion

A. Background

In chapter 3 you expanded the ideal gas simulation with particles that have larger diameters and experienced external forces. You found that, as they move through the box, the larger particles lose momentum to the smaller particles, which is the molecular basis of friction: the kinetic energy of the large particle is dissipated, and the solvent heats up. From a comparison of the inertial forces and the friction forces, the Reynolds number tells you that friction dominates the movement of small particles ($<100\ \mu m$) in a watery environment. It therefore takes a lot of effort to move small objects, like proteins and bacteria from one place to another. That raises the question: how can a cell manage to distribute all material required for its metabolism in time? Think about metabolites such as ATP, sugar or oxygen that are required for essential processes.

In the simulations we ran into another problem: making our simulations more realistic increased the computation time. Expanding the simulation further will quickly run our computer out of resources. You may argue that a faster computer will solve this problem, but there are no computers capable of simulating the entire contents of even the smallest cell. Compounding this situation, we've also run into the limitation that the simulation can only handle the simultaneous collision of two particles. When more particles meet, we will need to include more complex (and thus computationally expensive) physics. Taking smaller time steps may alleviate the multi-body collision problem but at the cost of more intensive computations. To get any further, we will need to abandon the idea of including all particles in the simulation, while still keeping an eye on the bigger biological problem:

How can cells distribute their metabolites efficiently despite the fact that friction annihilates directional motion?

B. Theory

In this section you should:

- get insight into the transport phenomena in cells

- familiarize yourself with the Galton board as a model for a 1-dimensional random walker
- derive Fick's laws and the diffusion equation

1 Cellular commuters

Cells have an advanced network of microtubules that form a track on which cellular transport takes place, see figure 4.1. As the name implies the microtubules are hollow tubes, made of tubulin dimers, that are 4 nanometer in diameter each.

Question B.1a What is the typical size, i.e. diameter and length, of microtubules?

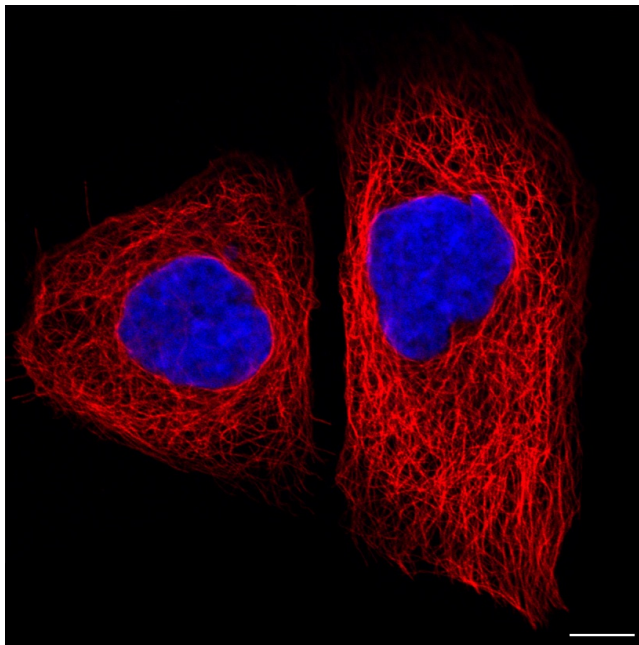


Figure 4.1: Cells contains an expansive network of microtubules (red). DNA is labelled blue. Scale bar 10 μm (From <https://v15.proteinatlas.org/>)

Like our train system, there are locomotives that follow the track and drag along cargo. Unlike our train system, the track has a directionality, with the minus end typically originating from the center of the cell and the plus end pointing towards the periphery. Specialized motor proteins called kinesins move along the microtubules to the plus end. Others, like dyneins, move in the opposite direction. Both typically carry a load, which may be a spherical vesicle with a diameter of several hundreds of nanometers. The spherical vesicle can be filled with cargo, for example neurotransmitters, that need to be transported to the edge of the cell.

A kinesin motor makes steps of 8 nm, and can take up to 100 steps per second. However, the velocity is not constant and the step itself is made in about 5 μs . This is an impressive speed, especially considering that the viscosity for such large particles inside the cell is about 5 times higher than that of water.

Question B.1b Calculate the friction force that the kinesin motor needs to overcome during the power stroke when it drags a 100 nm vesicle along the track.

Question B.1c How much work does this kinesin motor do per step? Express the work in units of $k_B T$.

Question B.1d The fuel for kinesin is ATP. If kinesin consumes 1 ATP molecule per step, what fraction of its energy is actually used for performing work?
(Tip: you have looked up the energy carried by 1 ATP molecule in chapter 1.)

Question B.1e How much ATP is needed to bring the vesicle 25 μm into the cell periphery? How long does it take?

Alternatively, the cell can just wait until molecules that are produced in the center of the cell move by Brownian motion. Take a small molecule with a radius of 0.5 nm. The diffusion coefficient for small molecules inside the cell is similar to that of water.

Question B.1f Use the Stokes-Einstein relation (eq. 3.9) to calculate the diffusion coefficient D for this molecule.

The mean squared displacement is proportional to D and time. Equation 3.8, was derived for 1 dimensional displacements. For n_{dim} dimensions it yields:

$$\langle x^2 \rangle = n_{dim} 2Dt \quad (4.1)$$

Question B.1g How much time does it take on average for a molecule to diffuse 25 μm in a (quasi) 2-dimensional cell?

Thus, the cell has two options to transport molecules: just wait for diffusion or use active transportation. The first works quite efficient for small molecules and distances on the scale of the cell and does not require any energy at all.

2 A drunkard's Walk

At the end of the previous chapter you have learned that the many collisions with small molecules displace larger molecules up to an extend that the displacement can be visualized. The actual direction of the displacement depends on the number, magnitude and direction of the colliding solvent molecules, which vary continuously due to the stochastic thermal motion. Because of this, the trajectory of the larger particle follows the prototype random walk: there is no memory and each next step can take any direction. Using the statistics of a random walk to describe the trajectory, we can omit the complex and time-consuming molecular dynamics simulation that computes the positions and velocities of all solvent molecules.

To visualize and quantify the behaviour of a random walk it is informative to

start with the Galton board. It is a device that forces a large number of particles through a regular maze of obstacles, such that each particle has an equal probability to pass the obstacle at the left or at the right side. The questions below are designed to take you through the statistics in very small steps (they are not trick questions!).

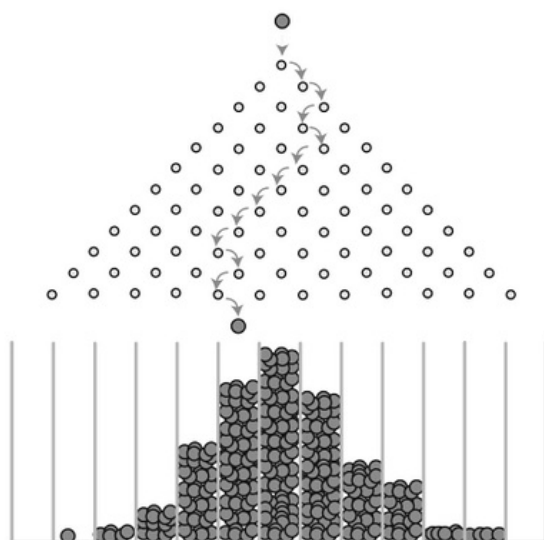


Figure 4.2: A Galton board (From <https://www.alanzucconi.com/>)

Question B.2a At how many different locations can the beads end up when they move through 4 layers of obstacles?

Question B.2b If the probability of passing an obstacle on either side is equal, what is the probability that a bead ends up in the most left bin?

The probability to end in a central bin is much larger, as you can see in figure 4.2. This is because there are more paths that lead to the central bin than to the left-most bin.

Question B.2c Write out all the sequences of choices, or paths, that end up in each of the bins for a Galton board with 4 layers. Use L for going left and R for going right. The left bin thus follows the sequence: $LLLL$.

The number of paths increases rapidly as the number of layers increases. Rather than writing out all possible paths, we want to use combinatorics to calculate the number of different paths, given the number of layers. Calculating how many different paths lead to the second bin in the Galton board then translates to how many options are there to distribute a single R in a 4 letter word containing only R s and L s.

Question B.2d Use a binomial coefficient notation to write down the number of paths for each bin, and calculate the outcome.

To convert the distribution of the number of paths to a probability distribution you need to divide the first by the total number of different paths. Of course you could sum the number of paths that you just calculated, but you can also calculate it directly, knowing the number of layers.

Question B.2e What is the formula for the total number of different paths?

The probability distribution that you just found is known as the binomial distribution. In a more general form it is defined as:

$$P(M; N, p) = \binom{N}{M} p^M (1 - p)^{N-M} \quad (4.2)$$

Question B.2f What do the parameters N , M and p represent for the case of the Galton board? Show that for equal odds of going left or right, equation 4.2 is indeed the ratio between between the number of paths leading to a particular bin and the total number of paths in the Galton board.

3 Fick's laws

You can even make more progress by ignoring the path of individual particles and only consider how groups of particles shift position over time. In this question you will derive the equation that describes all diffusive processes. Again, you will do this step by step.

Take the now familiar box with particles and split it up in sections, spaced by a distance Δx . Note that this is quite like the histograms that you made in the programming exercises. We are interested in the number of particles that pass through the imaginary boundary i that separates the section located at position x and the next section, located at position $x + \Delta x$.

Of course, this number will be proportional to the height of the box: when you make the box twice as tall, and given the same concentration of particles, twice as many particles N_i will cross the virtual section boundary i . For a real, 3-dimensional box, the number of particles passing this boundary will scale with the area of the boundary A . When you divide the number of particles passing this boundary by the area of the boundary, you get a parameter that does not depend on the dimensions of the box: the flux J_i .

Following the example of the Galton board, in time step Δt half of the particles will move to the left section and half of the particles will move to the right. The net flux through boundary i will depend on the number of particles in the section and the number of particles in the next section:

$$J_i = -\frac{\frac{1}{2}N(x + \Delta x) - \frac{1}{2}N(x)}{\Delta t A} = -\frac{\Delta N}{2\Delta t A} \quad (4.3)$$

Note that particles moving to the left will decrease the flux, particles moving to the right will increase it.

Question B.3a Show that when you multiply both counter and denominator with Δx^2 , you end up with:

$$J_i = -D \frac{\Delta N}{A \Delta x^2} \quad (4.4)$$

Question B.3b Express the flux J_i in terms of the concentration gradient $\Delta C / \Delta x$.

When you take the limit of $x \rightarrow 0$, Fick's first law comes out:

$$J(x) = -D \frac{\delta C(x)}{\delta x} \quad (4.5)$$

Fick's first law states that the flux is proportional to minus the concentration gradient (particles will move from high to low concentration), and the proportionality constant is the same diffusion constant as in equations 4.1 and 3.9.

For Fick's second law, consider how the number of particles in a single section changes in time, knowing the flux at its boundaries i and $i + 1$.

Question B.3c Express the change in the number of particles ΔN in one section in terms of the fluxes J_i and J_{i+1} and time Δt .

Question B.3d Rearrange this equation such that the difference between the fluxes, ΔJ is on one side of the equation.

Question B.3e Show that dividing both sides of the equation by Δx and taking the limit of $\Delta x \rightarrow 0$ and $\Delta t \rightarrow 0$ yields Fick's second law:

$$\frac{\delta C(x, t)}{\delta t} = - \frac{\delta J(x)}{\delta x} \quad (4.6)$$

Fick's second law says that the change in concentration in time is the opposite of the gradient in the flux. This makes sense, as particles will accumulate when the in-flux on one side of the section is bigger than the out-flux on the other side. In practice, it is difficult to measure the flux. One can eliminate the flux by combining both of Fick's laws into what is generally known as the diffusion equation:

$$\frac{\delta C(x, t)}{\delta t} = D \frac{\delta^2 C(x, t)}{\delta x^2} \quad (4.7)$$

, stating that the change in concentration in time will be proportional to the second derivative of the concentration and scales with the diffusion constant.

There are many solutions to this second-order differential equation. Each particular solution depends on the initial conditions and the boundary conditions. For N particles at $x = 0$ when $t = 0$ the solution is:

$$C(x, t) = \frac{N}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right) \quad (4.8)$$

Note that equation 4.8 has the shape of a normal distribution with a width of $2Dt$ and an amplitude that decreases with $1/\sqrt{t}$.

For more complex starting conditions, equation 4.7 may not be analytically solvable, but one can use numerical methods. In the spirit of the derivation of Fick's laws, one can compute the concentration at section x_i after time step Δt from the concentration distribution at the previous time step, assuming that all particles leave the section and half of the particles from both neighbouring sections enter it:

$$C(x_i, t + \Delta t) = \frac{1}{2}C(x_{i-1}, t) - C(x_i, t) + \frac{1}{2}C(x_{i+1}, t) \quad (4.9)$$

Question B.3f Use the central difference method, as depicted in figure 4.3, to show that the right-hand side of equation 4.9 is actually proportional to the second derivative of the concentration.

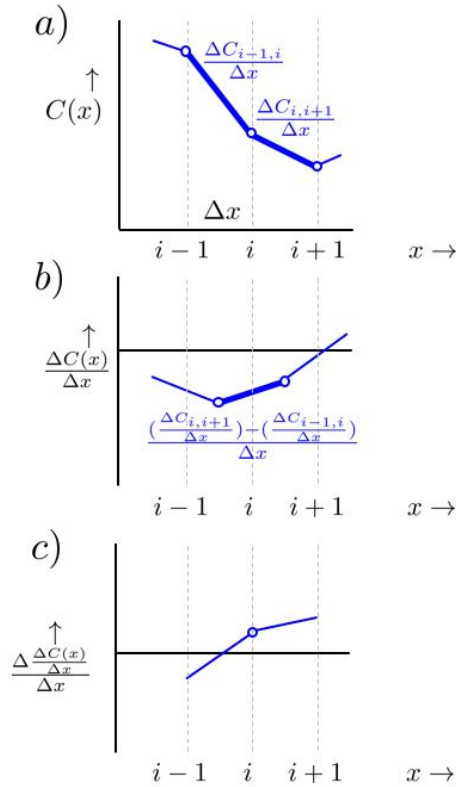


Figure 4.3: The central difference method to compute the second derivative from discrete points. a) The concentration plotted in discrete points. b) The derivative for each pair of points. c) The derivative of the derivative.

However, not all particles will leave the section for finite widths of the section. In fact, the fraction of particles that leaves will depend on the ratio between the average distance that a particle diffuses in time Δt and the width of the section

Δx . This leads to a discrete version of the diffusion equation:

$$C(x_i, t + \Delta t) = C(x_i, t) + \frac{2D\Delta t}{\Delta x^2} \left[\frac{1}{2}C(x_{i-1}, t) - C(x_i, t) + \frac{1}{2}C(x_{i+1}, t) \right] \quad (4.10)$$

Following the good programming practice to vectorize a problem, you can compute the term in the square brackets as `L*C[i-1:i+1]`, with the array `L=[0.5, -1, 0.5]`. The array `L` is known as the discrete Laplacian for 1 dimension. In the next section you will implement the numerical approach to find a solution of the diffusion equation in 2 dimensions.

Question B.3g What will the discrete Laplacian in 2 dimensions look like? (Hint: Follow the same reasoning that was used for 4.9)

C. Exercises

In this section you will:

- use random walks as a model for **Brownian motion**
- test **Central Limit theorem**
- simulate diffusion in **2 dimensions**

1 The Galton board

One of the premises of a Galton board is that each choice to move left or right is independent of the previous choice. In that case probabilities can be multiplied. you will use this to compute a large number of trajectories in parallel. Let's assume a board with 100 pins and 10 balls.

Question C.1a Use the `numpy` function `np.random.choice()` to generate a 2D array of points named `steps` with either -0.5 or 0.5. The first dimension represents the particles and the second the layers of the Galton board. Plot the steps of the first ball.
(Look up how the function `np.random.choice()` works if you don't know it)

Question C.1b Compute the trajectory of all balls in a single line of code using the function `np.cumsum()`. Plot the trajectories of all balls.
(Look up how the function `np.cumsum()` works if you don't know it)

Because you know the layout of the Galton board you know how to define an array `bins` such that when you use it in `np.histogram()`, each possible end position has its own bin.

Question C.1c Create a function `Galton_bins(n_layers)` that computes an `numpy` array `bins` accordingly. Print the result for a Galton board with 4 layers. How many elements does this array have?

A Galton board with 10 balls results in very poor statistics. You'll want to simulate many more trajectories. In addition, it is convenient to adjust the array `steps` such that the trajectories start at 0.

Question C.1d Make a function `Galton_board(n_balls, n_layers)` that has this modification. Test the function and plot the trajectories of a large number of balls.

Question C.1e Create a function `calc_pdf(x, bins)` that calculates the probability density function of `x` and returns both the `centers` of the bins and the corresponding `pdf`. Plot the beads' probability density function in the Galton board's last layer using this function.

Question C.1f Compare a scatter plot of the probability density function of layer 4, 16, 64 and 256 of a Galton board that contains 300 layers and 1000 balls with the theoretical expectation from equation 4.2. Plot the corresponding line plots in the same figure.
(Tip: It is convenient to use the function `binom()` from the module `scipy.special`.)

Question C.1g What happens to the amplitude of the pdf when you increase the number of layers by a factor 4? Which equation describes such behavior?

We used the Galton board as a proxy for a random walk representing a particle's movement in a solution of small molecules. The idea is that the random collisions with solvent molecules lead to a totally unpredictable displacement, but the random movement's statistical properties are deterministic. Moreover, as Einstein suggested, those statistics can reveal the properties of the underlying processes. Though the trajectory of the beads in the Galton board are not determined by diffusion, we can assign an effective diffusion coefficient to the mobility of the beads. Each layer would then correspond to a single time step Δt .

Question C.1h Calculate the squared displacement for each ball at each layer. Plot the squared displacement of the first 20 balls as a function of time in a single plot. Plot the mean of the squared displacement on top of it in black.

Question C.1i What is the slope of the mean squared displacement (MSD)?
What is the corresponding diffusion coefficient?
(Tip: You can use the function `np.polyfit(x,y,deg)` with `deg=1` to fit a line to the MSD.)

Question C.1j Make again a scatter plot of the probability density function of layers 4, 16, 64, and 256 of a Galton board. Now compare it with the solution of the diffusion equation for a point source (equation 4.8).
(Hint: Since eq. 4.8 is normalized, you can use $N = 1$.)
(Note: eq. 4.8 is a continuous function, you can plot the function with many more points than the number of bins to get a smooth curve.)

It may seem remarkable that two entirely different equations describe the Galton board equally well. However, you have encountered the underlying theory before: the central limit theorem which states that the sum of a large number of random variables (in this case the steps) follows a normal distribution, independent of the underlying probability distribution of the individual variable. In other words: you would have obtained a very similar result if you had chosen steps with a die (having 6 options for the step size), a continuous step size distribution, or even unequal steps or non-equal probabilities for steps. In the later cases, the mean would not be 0, but the distribution would still have the same shape. In the case of the binomial distribution, it has been proven by De Moivre that in the limit of a large number of steps the outcome is equal to a normal distribution. For other distributions, there is no mathematical proof but plenty of empirical evidence.

You could expand this simulation of random walks to 2, or more, dimensions, taking independent, random decisions for steps in each direction. Except that the MSD increases by a factor of 2 for each dimension (eq. 4.1), the physics is the same. Here, we conclude that the stochastic Brownian motion that characterizes the movement of individual particles can be captured in a simple formula that cannot predict the course of its trajectory but excels in describing its average properties.

2 Diffusion

Using the physics of random walks allowed you to capture the effects of collisions of a large number of solvent molecules into a simple sequence of randomly chosen steps. There is no need to do an extensive molecular dynamics simulation to obtain similar results, and by simulating random walks rather than Newtonian collisions you can gather much better statistics. Using Fick's laws we can go one step further. By ignoring the trajectories of individual particles and only looking at the net transport of species of molecules, we get closer to the familiar macroscopic world around us, at the cost of losing detailed insight into the behavior at the molecular scale.

Here you will simulate the effects of diffusion on the spatial distribution of the concentration of particular molecules. All the molecular properties of these molecules (like mass, size and shape) and their environment (viscosity) combine in a single parameter, the diffusion constant D . As in the molecular dy-

namics simulations, we will do the simulations in 2D. You will follow the same structure as you set up for the molecular dynamics simulation. Start by setting up the area in which the molecules can move.

Question C.2a Define a function `initialize(size)` which creates a square numpy array with a shape of `(size,size)`, in which the left half is filled with ones, and the right half is filled with zeroes. Plot a horizontal line to check the result.

```
def display_grid(grid, scale=None):
    plt.close() # close any previous plots
    fig, ax = plt.subplots(figsize=(3, 3)) # Adjust size
    ax.set_aspect('equal', 'box')
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    plt.xticks([])
    plt.yticks([])
    if scale is None:
        scale = [np.min(grid), np.max(grid)]
    im = ax.imshow(grid, cmap='hot', extent=[0, 1, 0, 1], \
        vmin=scale[0], vmax=scale[1])
    return fig, im
```

Listing 8: A small function to display a 2-dimensional grid.

Question C.2b Use the code in listing 8 to display the concentration in 2 dimensions.

To change the concentration in the array that you made with the function `initialize()` to a next time step, you can use the discrete formulation of the diffusion equation (eq. 4.10). This involves applying the Laplacian to each element in the concentration array, and adding the result to the initial array. You could do this in two nested for-loops, but by now you should be reluctant to use for-loops for large iterations.

Fortunately, the method of multiplying an array to each of the elements of another array is needed quite commonly, and is known as convolution. In the module `scipy.signal` you can find the function `convolve2d()` that does just that. The function works by sliding the kernel array (which would be the Laplacian) over the input array (the concentration profile), summing the product of elements that overlap and storing the result in the output array at the corresponding position. This process is repeated for every position in the input array, resulting in the convolved array. However, the implementation is faster than 2 nested for-loops.

Moreover, the function also has convenient ways to deal with the edges. The edges are problematic, in the sense that the left-most element of the input array does not have a neighbor on the left, which makes it impossible to compute the convolution. The function `convolve2d()` either discards the outer rows and columns of the array (`mode='valid'`), adds new rows and columns with the

outer values of the kernel (`mode='full'`) or yields a value for all elements of the input matrix (`mode='same'`). Because we want to iterate the procedure for every time step, we will use the latter, so our array will keep the same shape. In addition, there are options for dealing with the Laplacian when it runs into an element at the edges. These options are set by the parameter `boundary`.

Question C.2c Look up the options for the parameter `boundary` in the function `convolve2d()`. Relate two of these options to the boundary conditions that you used in the molecular dynamics simulation.
(Tip: If you cannot answer this question directly from the documentation, you may want to try these options later when the full simulation works and evaluate the result)

Question C.2d Create a function `advance(concentration, D=1, dt=1, kernel=None)` that applies the appropriate 2D discrete Laplacian to the concentration array that you made with the function `initialize()`. In case `kernel` is not set, use the 2D discrete Laplacian as discussed at the end of section B. Plot a single row of the result and compare with the same row of the input array `concentration`.

If you did it correctly, you will only see a very small change in `concentration`. You might want to use a larger diffusion constant or a larger time step. However, you can change the values of `D` and `dt`, only such that the product of both does not exceed 1.

Question C.2e Try the function `advance` with a larger `D` or `dt` and plot a row of the result. Use equation 4.10 to explain what happens.

Question C.2f Set up a simulation (using default values and the initial state of all molecules in the left half of the grid) for 1000 time steps and store every 10th result in a 3-dimensional array `concentrations`. Use the function `animate_grid()`, as given in listing 9, to display a movie of the result.
(Tip: You can use a similar structure as the function `demo_md()` in `MDsimulation.py` that you used in the previous chapter.)

The solution of the diffusion equation (eq. 4.7) depends of course on the initial conditions. The function `advance` is very flexible and any initial concentration profile will work. So let's aim to reproduce eq. 4.8, i.e. the situation in which all molecules start at the same location. Because you cannot make an infinitely small grid, you will start with an initial concentration profile that is very close to that.

Question C.2g Amend the function `initialize()` such that it accepts the parameters `profile` and `width`. Put an `if-elif` statement

```

def animate_grid(grids, frames=None, scale=None):
    # Put the call to this function at the end of the CoCalc cell
    def add_frame(i, grids, titles):
        im.set_data(grids[i])
        plt.title(f'frame {frames[i]}')
        return im

    fig, im = display_grid(grids[0], scale=scale)
    if frames is None:
        frames = range(len(grids))
    titles = [f'frame {frame}' for frame in frames]

    anim = animation.FuncAnimation(fig, add_frame, len(grids), \
        fargs=[grids, titles])
    for _ in tqdm(frames, desc='Animation'):
        anim._step()
    return HTML(anim.to_html5_video())

```

Listing 9: A function to create a movie of a 3-dimensional array `grids`.

in the body of the function, such that by default it creates the 'left-only' distribution that you made before, but that it creates a Gaussian profile with the provided `width` at the center of the grid if `profile` equals `'gaussian'`. Display the result for a width of approximately half the grid size.

Question C.2h

Do a 1000 step simulation starting with a Gaussian concentration profile with a width of 5% of the grid size and `dt=0.5`. Show an animation of the result. (Tip: if your simulation becomes unstable, you may need to adjust the input parameters of the function `advance()`)

Question C.2i

To get a better view of the result, plot a cross-section of the concentration profile through the central position (1-dimension is sufficient) for a couple of frames distributed equally over time.

Like in the case of the Galton board, you should check whether the mean squared displacement increases linearly in time. We don't know the positions of the molecules, so we cannot directly calculate their mean. However, we know that the probability distribution of finding a molecule is proportional to the concentration. Therefore, you can use the second moment of the positions of the molecules to calculate the mean squared displacement, see eq. 1.3. For this you need i) an array of the positions `x` and ii) an array of the probability distribution of the molecules corresponding to positions `x`. (It is sufficient to do this in 1 dimension)

Question C.2j

Create the array `x`. Make sure that the center of the concentration profile is located at `x=0`. Convert a horizontal

cross-section of the concentration profile at the center of the peak to a probability distribution and calculate the second moment of $\mathbf{x}$. Do this for every frame of the array `concentrations` and plot the MSD as a function of the frame number. Is it linear? Does it go through the origin? If not, can you explain why?

To demonstrate the versatility of the convolution method you can modify the convolution kernel a bit.

Question C.2k What would happen if you decrease element `[0,1]` to 0.15 and element `[2,1]` to 0.35? The sum of all elements should be 0. Why?

Question C.2l Run the simulation with this modified kernel and show the result in an animation.

Question C.2m To show the displacement of the molecules, plot the mean position as a function of the frame number. Use the first moment (eq. 1.2) to compute the mean x-position for each frame. What is the mean velocity of the molecules?

D. Test

1 Slowly drifting away ...

Question D.1a What parameter would you need to change in equation 4.2 to introduce a drift in the results of the Galton board (No need to do the simulation). How could you induce that situation with a real, physical Galton board?

Question D.1b Copy or repeat the simulation with the modified convolution kernel of the last exercise and plot the MSD as a function of the frame number. Is it linear? Why (not)?

Even when there is drift, it is possible to extract extend of diffusion. For that you need to compute the *central* moment, i.e. the moment of the difference between to parameter of interest and the average of that parameter. The second central moment is known as the variance, or the squared standard deviation, and can be calculated from the probability distribution:

$$\sigma_x^2 = \sum (x^2 - \langle x \rangle^2) \cdot P(x) \quad (4.11)$$

Question D.1c Plot the second central moment of the concentration profile. Is it linear? Is the diffusion constant the same in both directions?

2 Is there enough to eat?

Though diffusion can be fast for small molecules and small distances, it puts severe constraints on the cells metabolism. Here you will simulate the flux of molecules, like glucose, into the cell.

Question D.2a Create a grid of shape $(300, 300)$ filled with a (glucose) concentration of 1, except for a circle with a radius of 50, in which the concentration is 0. Display the resulting grid.

In this grid, the central circle represents a cell, and we enforce that the glucose is directly processed in the cell, resulting in a glucose concentration of 0.

Question D.2b Modify the code for the simulation such that after each call of the function `advance()` the concentration within the cell is set to zero. Run a simulation, and show the animation. Describe what happens.
(Tip: create a Boolean array `cell` whose contents is only `True` for those elements inside the cell. Use slicing with this array to select the concentration in the cell.)

In biology, homeostasis is the state of steady physical and chemical internal conditions maintained by living systems. The cell needs to have a continuous supply of nutrients, such as glucose, to maintain homeostasis. We want to see how much glucose enters the cell.

Question D.2c Modify your code such that you keep track of the total amount of glucose that enters the cell per iteration. Plot this glucose flux as a function of the frame number. What is the rate of glucose consumption in a stable situation?

Since the intake will scale with the surface area of a cell and the required nutrients are likely to scale with its volume, the diffusion of nutrients sets a limit to the maximal size of a (clump of) cells. If a cell needs more, the only solution is to mix the solution, such that the depleted volume around the cell is replaced with a solution of fresh nutrients. This strategy may be movement of the cell itself for mono cellular organisms, or the use of veins that bring in a fresh supply of nutritious blood in multi-cellular organisms.

Question D.2d Modify the convolution kernel such that there is a flow of the solution. Animate the result. What happens around the cell?)

Question D.2e Show that this can significantly increase the influx of glucose by plotting the influx as a function of frame number with and without flow in the same plot.

Chapter 5

Entropy

A. Background

In the first chapter, you have seen that cells contain a huge number of molecules, most of which are water and that the large molecules that actually organize life, like proteins and DNA, have structures that can sometimes be measured with sub-nanometer precision. The latter suggests an exquisitely well-defined structure of these polymers. In the next chapters, you studied the behavior of small, non-interacting molecules and learned that they move around at very large speeds. Moreover, frequent molecular collisions ensure that not only the positions of the molecules continuously change, but also their energies continuously vary when momentum is exchanged. The consequence was that the positions and energies of individual molecules are ill-defined but that their distributions are firmly set by the boundaries of their confinement and the Maxwell-Boltzmann equation respectively. Finally, looking at diffusion, you've noticed that in time, all molecules spread out such that concentration gradients cease to exist.

Somehow bio-molecules seem to defy this tendency for disorder and they can even assemble into magnificent larger structures like organelles and whole cells. It appears that we are further away from explaining the physics of life processes than before we started. One could question whether the ideal gas is of any help at all to understand biological phenomena. If only, because the major constituent of cells, water, hardly qualifies for the criterion of non-interacting molecules: the fact that water is liquid testifies to the strong interaction between water molecules.

Here, we will look more thoroughly into this universal tendency to increase disorder as stated by the second law of thermodynamics which says, in simple terms, that entropy always increases. Paradoxically enough, it is exactly this principle that ensures the precise folding of proteins, the highly selective interactions between molecules, and even the production of highly energetic molecules that the cell uses as fuel. So the central question of this chapter is:

How to reconcile the tendency to increase disorder, as stated by the second law of thermodynamics, with the highly controlled and structured processes in life?

B. Theory

In this section you will:

- quantify the amount of disorder into a single parameter: entropy
- relate entropy to the distribution of the positions and energies of molecules in a (micro-)canonical system
- evaluate how entropy plays a defining role in biomolecular interactions

1 How to define a complete mess?

To quantify the amount of (dis-)order let's first go back to the 1-dimensional random walk, like that of a single ball in a Galton board. The trajectory of a single particle is fully described by a string of letters, for example, *LLRRLLRLL*, representing a step to the left or to the right. Without any compression of the data, each step requires a single bit (0 or 1), when you want to store it in a file on your hard disk. Let's call the number of bits required for storing the data stream the information content I . If you have more than 2 choices per event it will require more bits to store all information:

$$I = \log_2 \Omega = K \ln \Omega \quad (5.1)$$

with Ω the number of different strings, which would be equal to M^N for N events and M choices (and ignoring permutations!).

Question B.1a DNA has 4 different bases. How many bits does it take to store all the information of a 120 bp gene?

Question B.1b How much information is in the amino acid sequence of the corresponding protein? Explain why it is the same/different.

Question B.1c Parameter K in eq. 5.1 is a constant. What is its value?

When you tilt the Galton board, you will more likely observe trajectories like *LLLLLLRLLL*. The trajectory has become much more predictable, as most steps go to the left. Without losing any information, you can save disk space by storing only the layer numbers in which the ball took a step to the right. The longer the stretches of steps to the left, the less information you need to keep. Indeed, the information content is less and should be adjusted for the permutations in the string that don't change the outcome:

$$\Omega = \frac{N!}{N_1! N_2! \dots N_M!} \quad (5.2)$$

Question B.1d Recalculate the information content of both 120 bp DNA and the corresponding amino acid chain using eq. 5.2 and assuming that all types of bases and amino acids are equally distributed. Why are both answers different than before?

A more common definition of the information content was stated by Claude Shannon:

$$I = -N \sum_{j=1}^M p_j \ln p_j \quad (5.3)$$

, where p_j equals N_j/N .

Question B.1e Use the Stirling approximation, i.e. $\ln N! \approx N \ln N - N$ for large N , to show that the definition for information content in eq. 5.1 is equal to the Shannon information.

Question B.1f Calculate the information content for trajectory in a ‘perfect’ Galton board of 10 layers and show that the information content goes down if there is a bias in the probabilities. To do so, make a sketch of $I(p)$ for $0 < p < 1$. (Tip: No need for an analytical derivation: just calculate $I(p)$ for a few distributed values of p .)

2 Boxed up chaos

The information content analysis from the previous paragraph is also useful for defining the amount of (dis-)order in our molecular dynamics simulation. However, in physical problems it is more common to define the amount of disorder as the entropy S :

$$S = k_B \ln \Omega \quad (5.4)$$

Question B.2a How are S and I related?

To fully define the state of the particles in the box (like if you would want to stop the simulation halfway, store the data, and continue the simulation some other time), you would need to save all the positions and the velocities of all particles. In terms of information content, it’s important to find out how accurate you need to know these parameters: more precision means more bits. You’ll also need more bits if the box or the velocities span a larger range. Below, you will work towards an equation that takes into account all these parameters, and end up with a single number for the amount of disorder in the box, or equivalently the number of bits that you need to store all the information about the state of the box.

It’s good to reiterate how we set up the box as a closed system: particles cannot enter or leave the box, the box does not change in volume, nor is there exchange of energy with its surroundings. Such a closed system, defined by the number

of particles N , its volume V , and its total energy E is called a micro-canonical system in the field of thermodynamics and you are looking for an expression of $S(N, E, V)$.

Let's first consider a single particle in a 3-dimensional box. If you split the box up into small cubicles with edges of size Δx , corresponding to the accuracy with which you know the position, you could use 1 bit for each cubicle to tell if the particle is in the cubicle or not.

Question B.2b What is the expression of $S(V)$ for one particle in the box for an accuracy of Δx ?
(Hint: Use the number of different, distinguishable positions that the particle can choose to compute the entropy.)

When considering multiple particles that collide, we used conservation of momentum to compute the new trajectories. However, let's first consider the momentum of a single particle.

Question B.2c Express the energy E of a single particle as the sum of its momentum in 3 directions.

Question B.2d If you consider the momentum of the particle as a vector that points in an arbitrary direction, what is the radius of the sphere that describes all possible momentum vectors in 3 dimensions as a function of its energy?

To include the momentum in the calculation of the entropy of the system, you first need to find the number of distinguishable momentum vectors of the particle, given an accuracy Δp . As for the position, this comes down to counting the number of cubicles of size Δp^3 (remember that $\vec{p}$ can point in 3 directions) that fit in a shell with a radius $|p|$ and a thickness Δp . Surely, this number will scale with the surface area of the sphere.

Question B.2e Use the formula of the surface area of a sphere to express the entropy of the momentum of a single particle in terms of its energy: $S(E)$. You don't need to express factors Δp in terms of E .

Question B.2f Combine the terms $S(V)$ and $S(E)$ into a single expression $S(E, V)$.

How accurate do you need to know the position and the momentum? Quantum mechanics tells us that it's impossible to measure the position or the momentum with infinite precision. In fact, the accuracy of the product of the two is limited, as stated by Heisenberg's uncertainty principle: $\Delta x \Delta p \geq \frac{\hbar}{2}$. This sets a limit for how accurately we need to define our system, and thus for the size of the entropy.

Question B.2g Use Heisenberg's uncertainty principle to eliminate Δx in your previous answer.

The situation for more than one particle is more complex. The positions of all particles are independent from each other and the contribution of the positions to the entropy will therefore simply scale with the number of particles N .

The momentum of each particle, however, is not independent of the momentum of the others. On the contrary: the sum of all the momenta should remain constant in our micro-canonical system, so when one particle slows down, the other particles need to speed up. But how this change of momentum of the other particles is distributed is unknown. Whereas the possible momentum vectors of a single particle in the system all end on a sphere with a radius that is defined by the energy, for multiple particles the radius of these spheres will vary within the restriction that the sum of the radii is set by the energy.

For two particles, one could envision this as two filled balloons that are connected by a tube: when one shrinks, the other expands (in this analogy however, the volume is maintained, rather than the sum of the radii). Another way to deal with the coupled momentum vectors is to put both vectors in the same sphere. You'll need 6 dimensions though, to define all degrees of freedom in terms of momentum. In this 6-dimensional space, the joint momentum vector can take any direction, but its magnitude is constrained to a radius set by the total energy of the system. So for N particles, you'll need a $3N$ -dimensional sphere. Taken together, (and making a small assumption to simplify the surface area of a $3N$ -dimensional sphere) the entropy of the micro-canonical system with N particles follows the Sackur-Tetrode formula:

$$S(N, V, E) = k_B [\ln (A_{3N}(2mE)^{3N/2}) + \ln V^N - \ln N! - \ln (2\pi\hbar)^{3N}] \quad (5.5)$$

, in which A_{3N} represents the area of a unit sphere in $3N$ dimensions. You will find appropriate values for this parameter in the exercises. Equation 5.5 looks complicated, but by now you should be able to identify each term in the square brackets. There are 4 contributions to the disorder: uncertainty of the position, uncertainty of the momentum, quantum mechanical uncertainty, and a term $-\ln N!$.

Question B.2h Explain in one or two sentences what the term $\ln N!$ in the Sackur-Tetrode formula represents.

The absolute value of the entropy is generally of little use: it is for most problems sufficient to quantify changes in entropy. For this, the Sackur-Tetrode formula has the very convenient property that the four contributions are additive, so if only one system parameter changes, one only needs to evaluate part of the equation.

Question B.2i If all N particles are at first constraint to the left part of the box, for example by an imaginary central wall, how much does the entropy change when this constraint is removed? How much more information is required to define the system's state?

Now imagine that you have two kinds of particles in the split box. They don't interact, but the wall between the two compartments is selective and only stops

the first kind of particles. The second type is free to pass the boundary. A sieve would be a good metaphor for such a semi-permeable wall: small particles can pass, but larger ones are reflected. Even if you start with both types of particles on the left side of the box, after some time the smaller particles will spread out like the wall does not exist. The larger particles will remain constrained to the left part as long as the wall is there.

Question B.2j Argue how your answer to the previous question would change/stay the same for the larger particles. Use the Sackur-Tetrode equation to make your point.

The membranes that separate organelles in cells, and cells from their environment, are actually semi-permeable: most of the contents of the cell cannot pass the membranes, but water molecules can. When evaluating the entropy of the water molecules, it's like the membranes are not there. Consequently, when evaluating any processes in the cells, the entropy of the water molecules will not change by the confinement and therefore will not contribute to a change in the total entropy. As you will see later, there are other, more subtle entropy-related effects connected to water that do make a major difference. For now, it is important to realize the consequence of the semi-permeable property of the membranes: as far as entropy is concerned we can just ignore the presence of 75% of the cell contents and focus on the remaining constituents. These are free to move around within the confinement, as if in a vacuum, except for the friction forces that mediate and speed up the exchange of momentum between particles.

Note that the choice of the starting state gives direction to the process described above: it is possible, but extremely unlikely, that all molecules will spontaneously return to a position on the left side of the box. This is the principle underlying the second law of thermodynamics: all positions (and momenta) are equally likely to be occupied. Any difference between the actual distribution and those defined by the size of the box and Maxwell-Boltzmann implies that the distributions are not stable and will reach the equilibrium distributions when given more time.

3 Get real: open up!

It's time to get back to our original objective: how to describe the behavior of a mixture of real molecules in the real world, rather than identical, ideal gas-like particles in a closed system. For a more realistic description, we need to take into account the exchange of energy and material with the environment. Such situations are referred to as canonical and grand-canonical systems respectively. In this course, we will only look at the first, canonical systems, in which the exchange of energy is possible. Very similar arguments can however be made for the exchange of particles in grand-canonical systems.

By allowing an exchange of energy, we can define sub-systems that resemble real-world structures like single molecules, organelles, or cells. These are relatively small systems as compared to their bulky environment but still contain a large number of atoms to keep track of. The environment is of course many times larger than the small sub-system and this means that any exchange of energy will hardly affect the total energy of the environment, but may have a large

impact on the much smaller sub-system. Since energy and entropy are extensive variables (they double in magnitude when the system doubles in size) it will be impossible to account for the entropy and energy of the entire environment. Therefore simple conservation of energy or maximization of the total entropy are not viable strategies to learn more about the sub-system. Temperature on the other hand is an intensive variable (it doesn't change with the size of the system) and is closely related to energy. Moreover, the temperature of the environment is easy to measure and control.

From the Sackur-Tetrode equation (eq. 5.5) it follows that the temperature of the system is defined by:

$$T = \left(\frac{dS}{dE}\right)^{-1} \quad (5.6)$$

Question B.3a Use of the equipartition theorem to prove this definition of temperature.

Consider a small sub-system a that is surrounded by a much larger system b . The second law of thermodynamics says the entropy of the total system can not decrease:

$$\Delta S_{tot} \geq 0 \quad (5.7)$$

But what happens when some energy is exchanged between the two systems? Sub-system a is small enough to evaluate both ΔE_a and ΔS_a . The energy that enters system b must be the same as the energy that left sub-system a : $\Delta E_b = -\Delta E_a$. But how about the change in entropy for system b ? Entropy is not conserved (if anything, it increases!) so you can not apply a similar conservation argument. However, using eq. 5.6 one can express the change in entropy of system b in terms of its change in energy: $\Delta S_b = T \Delta E_b$.

Question B.3b Finish the above reasoning and derive a formula for ΔS_{tot} that only contains properties of sub-system a .

This is a remarkable result: though there is extensive contact between sub-system a and its environment, the only way system b enters the equation of the change of the total entropy is through its temperature. And because of its large size, its temperature will not change. So both the effects of exchange of momentum and the tendency to distribute the positions and velocities of the entire system b as much as physical constraints and Maxwell-Boltzmann allow are all summarized in this single constant parameter T .

To emphasize that it is really all about the properties of sub-system a one can use the formalism of the Helmholtz free energy:

$$F_a \equiv E_a - TS_a \quad (5.8)$$

Question B.3c How does the Helmholtz free energy relate to the total entropy S_{tot} ? Will F_a also be maximal when the total system is in equilibrium?

Note that the energy, E_a , includes chemical energy. When molecules are involved in a chemical reaction, rearranging their atoms may cost or release

energy which will then directly be extracted or inserted in the sub-system, as energy is conserved. Still, chemical reactions will tend to increase disorder either by heating up the system or by increasing the variation in positioning of the molecules by increasing the number of different species of molecules.

It's good to realize the meaning of the word *free* in the term free energy. The term *free* reflects the fact that not all the energy in sub-system *a* is available for other processes: part of the internal energy will always be associated with making sure that the particles in sub-system *a* will have the same temperature as that of the system *b*. Whatever part of the energy that remains can be used for other processes, for example, to expand the volume of sub-system *a*. Increasing the volume takes an amount of work $p\Delta V$. This is relevant for many chemical reactions in which gasses are produced or taken out of the system and a second type of free energy has been introduced for this situation: the Gibbs free energy:

$$G_a \equiv E_a + pV - TS_a \quad (5.9)$$

In biology, most reactions take place in water and do not involve gasses, reducing the work term to zero, and making the Helmholtz and the Gibbs free energy equal. Nevertheless, most literature uses the term Gibbs free energy even though work does not play a role.

You are now fully equipped to understand many seemingly paradoxical processes that take place in biological matter. **The key to better understanding life processes is to realize that two entirely different processes can be coupled, such that the net change in entropy, or free energy, for the entire process is favorable.** Often only one of the two coupled processes is highlighted, which creates the wrong impression of an apparent increase in order.

For example: the kinesin motor proteins that drag vesicles through the cell perform a lot of work. Moreover, they can deliver cargo to well-defined places in the cell, working against a concentration gradient and thus creating more order. However, they can only do that because ATP is simultaneously converted to ADP and phosphate. This reaction has two implications for the change in free energy: i) the energy (in this case the chemical energy in the bond between ADP and phosphate) in the sub-system is reduced and ii) the entropy is increased because there are now two independent particles instead of one.

Question B.3d For cells it is essential to keep the concentration of ADP low and the concentration of ATP high. Why does this affect the free energy of the reaction?

4 Entropy rulez...

There are many biological processes that seemingly create order where increasing entropy is the driving force. For example, large molecules, like proteins interact in a very specific way and form intricate structures. Such structures assemble spontaneously, apparently despite the second law of thermodynamics. However, the complete picture is consistent, when taking into account all the other molecules in the crowded cell: assembly of the large structures leaves more

space for other molecules to increase their entropy. The mechanism that brings these large structures together is referred to as the *depletion interaction*.

To make this point more clear, imagine a box filled with two large cubicle particles with sides l and many smaller spherical particles with radius r . Even in the absence of an interaction energy between the square-like particles, entropy will make them stick together when a sufficiently large number of smaller spherical particles are confined in the same box.

Question B.4a How much more volume is available for the spherical particles when the two cubicle particles come together as compared to the situation when the two cubical particles are apart?

To calculate the difference in free energy for aggregation of the cubicle particles, it is convenient to relate the free energy to a pressure that is applied by the smaller spheres on the blocks.

Question B.4b Show that the pressure can be calculated from the derivative of the free energy to the volume, i.e. $\frac{dF}{dV} = -p$, by combining equations 5.5 and 5.9 and 3.10. Explain that the minus sign makes sense.

The number of large, cubicle particles (2 is enough) is much smaller than the number of intermediate-size spherical particles, so the change of entropy due to their aggregation will be negligible. As argued before, the solvent molecules, like water, can be ignored, because the volume in which they can move freely does not change when the larger particles come together. This is not the case for the spherical particles with their intermediate size. The concentration of these spherical particles in the box is just N/V and plugging that into the result you just obtained (or into the ideal gas law), yields the van 't Hoff relation:

$$p_{\text{equil}} = c k_B T \quad (5.10)$$

The index 'equil' reminds us that the van 't Hoff relation is only applicable when the system has come to a stable situation. Though it may be awkward to think of pressure in a liquid, the analogy with the ideal gas is there: the spherical particles exchange momentum with the walls of the confinement, and by doing so exert a force on the walls. The pressure on the walls is an osmotic pressure, that would cancel if both sides of the semi-permeable wall contained the same concentration of dissolved particles. If the space that is confined by the semi-permeable wall contains a higher concentration, the particles push the semipermeable wall towards the lower concentration side. As a side note: this is what gives rigidity to plants.

Coming back to the depletion interaction: the void in between the two cubicle particles will be depleted from spherical particles when the gap becomes smaller than the diameter of the spheres. This will create a negative pressure (the pressure on the other sides of the cubical particles is larger than the pressure on the two sides that face each other) and will bring them together.

Question B.4c Show that the difference in free energy between the situation when the gap is just small enough to exclude the

spheres and the situation when the gap is fully closed equals: $\Delta F = -c k_B T 2r l^2$.

Now let's see if the magnitude of the depletion interaction is significant. If the difference in free energy is smaller than $k_B T$, then thermal fluctuations will overrule the interaction. If the difference in free energy is larger than $k_B T$, then it will be stable. Reasonable numbers for the sizes of the particles are $r = 2.5$ nm, $L = 15$ nm.

Question B.4d How many spherical particles do you need to drive the cubicle particles together in a $1 \mu m^3$ box? Is it reasonable to assume such concentrations in cells?

Of course, modeling intricate structures like proteins as cubic or spherical particles is a gross simplification. Nevertheless, the mechanism that you just discovered does not depend on the precise geometry of the structures. It's just easier to do calculations with cubes. There is also an important experimental implication: studying interactions between proteins and/or DNA in a test tube, rather than inside a cell, will underestimate the strength of attraction between them unless inert crowding molecules are provided with a size and concentration similar to that of the constituents in the cell.

C. Exercises

In this section you will:

- get a better insight into higher dimensional space
- get acquainted with Monte Carlo simulations
- learn more about the thermodynamics of molecular motors

1 A piece of pi

The Sackur-Tetrode equation contains a term that represents the area of a unit sphere in 3N-dimensional space. This is hard to imagine, especially because N , the number of particles can get very big. But even for small numbers of N it is not trivial to compute properties like its surface or its volume. For 3 dimensions we all know the surface of a sphere is $4\pi r^2$. But how do you get the value of π if you cannot just look it up?

Here we will first numerically compute the value of π using a method called Monte Carlo simulations and then compute the surface area of spheres in higher dimensions. The name stems from the city that hosts many casinos and the method relies on handling probabilities, like throwing dice. Casino owners make sure that their enterprise will on average make a decent profit, but they also know that the profit can fluctuate a lot when the number of customers gets small.

The strategy to compute the value of π is to compare the area of a circle with the area of a square that tightly bounds the circle.

- Question C.1a** What is the ratio between the area of a circle with radius $r = 1$ and the area of the square around it?
- Question C.1b** Generate a large `numpy` array of shape `(n,n_dims)` filled with random values between -1 and 1. Use `n_dims=2` and `n=1e4`. Check the result.
- Question C.1c** Calculate the number of points that have a distance to the origin smaller than r^2 . Use a single line of code.
- Question C.1d** Use this number to obtain a value for π . What is the difference between the value that you obtained and the value that `numpy` provides?
- Question C.1e** Put the code to calculate π into a function `calc_pi(n_dims,n=10000)` and make sure that it gives a good approximation for `n_dims=2` or `3`, but returns `np.nan` otherwise. Check the function by printing the results for `n_dims = 0..4`.

Of course, the numerical estimation of π increases in accuracy as you evaluate more samples.

- Question C.1f** Run the simulation a couple of times and calculate the mean and the standard deviation of the result for `n=10000`. Use list comprehension to keep the code clean and fast.
- Question C.1g** Compare the standard deviation as a function of the number of samples. Let `n` vary over at least 4 orders of magnitude. Plot the standard deviation as a function of `n` on a log-log plot.

If all goes well, your plot is a straight line.

- Question C.1h** What is the slope? What does this tell you about the relation between `n` and the standard deviation of your simulation of π ?

2 A heat engine

In section B you learned that:

- any process should obey the second law of thermodynamics, stating that entropy never decreases.
- the starting situation gives direction to the process

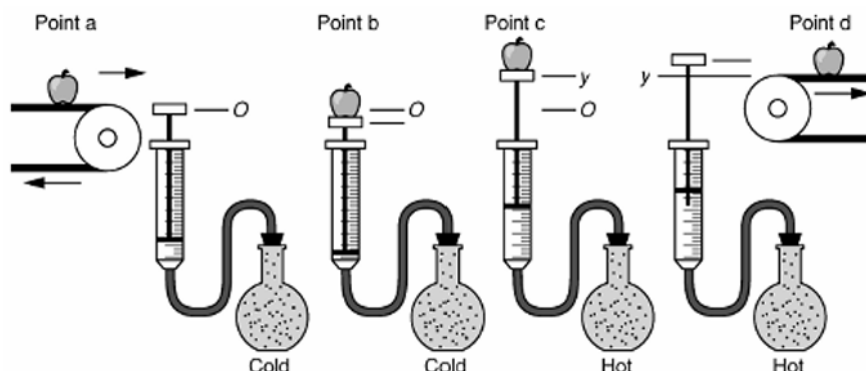


Figure 5.1: A heat engine performs work while cycling between heating and cooling the gas in a piston that is connected to a flask. The flask is alternately inserted into a cold bath and a hot bath (not drawn, but depicted by the words *Cold* and *Hot*) which change the temperature of the gas. An apple is shifted onto the piston that contains cold gas (a and b). When the flask is put into a hot bath the gas expands (c) and the apple is raised to a higher level, from which it can be shifted sideways again (d). Putting the flask in a cold bath will reduce the volume of the gas and bring the position back to the starting level (a). After a full cycle, the engine performed an amount of work that corresponds to the increased potential energy of the apple (ignoring any friction during the lateral moves), and some heat is exchanged between the hot and the cold bath. (Figure from <http://galileo.phys.virginia.edu/classes/142w.rmm5a.spring04/manual/Lab13.pdf>)

- free energy accounts for the entropy change of the outside world by enforcing a fixed temperature to a smaller, thermally coupled system
- a process that decreases entropy can still proceed spontaneously, but only if it's coupled to another process that increases entropy by at least the same amount

In this set of questions, you will explore how to use these concepts to make a machine that actually does work. The machine does the seemingly impossible: it will use heat as an energy source and will move weight against gravity, effectively converting thermal energy into potential energy, see figure 5.1. Note that combustion motors and steam engines essentially follow the same mechanism, the only difference being that the heat is provided by burning fuel or coal rather than two baths. Motor proteins in the cell, like kinesin, don't use the expanding gas as an intermediate, but directly change conformation upon hydrolyzing its ATP fuel.

Side note: You will not use a lot of code in this question, but you are asked to provide some formulas. You can toggle the input of each cell in your `IPython Notebook` between `code` and `markdown`. In `markdown` mode you can input text and `LaTeX` code. For `LaTeX` you need to put your strings between a pair of single or double `'$'` characters. In this way, you can insert beautifully formatted formulas and/or Greek characters. You can use the same pair of `'$'` characters to put `LaTeX` formatted strings in `Matplotlib` plots. This question may be a good opportunity to practice with `LaTeX` formatting using. If you are not familiar with `LaTeX` you can find formatting codes in this cheat sheet.

In the cycle depicted in figure 5.1 energy is exchanged between the gas and the two baths in two steps. When the warm flask is put into the cold bath (d→ a) an amount of heat Q is transferred from the flask to the bath, reducing the volume of the gas. During the power stroke (b→ c) the opposite happens: the same amount of heat Q is taken from the hot bath to increase the volume of the gas. But that's not all: the apple is also lifted. Because energy is conserved, the energy corresponding to the work W to lift the apple also needs to be extracted from the hot bath. Using equation 5.6, the change of entropy can be described as:

$$\begin{aligned}\Delta S &= \frac{\Delta E_{shrink}}{T_{cold}} - \frac{\Delta E_{powerstroke}}{T_{hot}} \\ &= \frac{Q}{T_{cold}} - \frac{Q+W}{T_{hot}} = Q\left[\frac{1}{T_{cold}} - \frac{1}{T_{hot}}\right] - \frac{W}{T_{hot}}\end{aligned}\quad (5.11)$$

Question C.2a Use equation 5.11 to explain why the heat engine can only work if $T_{hot} > T_{cold}$.

Question C.2b Use equation 5.11 to explain why the heat engine works better when $T_{hot} \gg T_{cold}$.

So the heat engine works optimally when the temperature difference is very large. However, this is not an option in a cell, if only because water would start to boil or proteins would start to denature. However, there is another option to increase the efficiency. The efficiency η of the power stroke of the engine can be defined as:

$$\eta = \frac{W}{|\Delta F|} \quad (5.12)$$

Question C.2c Why is it necessary to take the absolute value of ΔF ?

Question C.2d Express the work as a function of the initial volume V_i , the final volume V_f and the pressure p .

For this exercise, it is useful to define the relative step size δV^* :

$$\delta V^* = \frac{V_f - V_i}{V_f} = 1 - \frac{V_i}{V_f} \quad (5.13)$$

Question C.2e Rewrite your result in terms of the relative step size δV^* and the final volume V_f .

Question C.2f Calculate the change in free energy as a function of the relative step size δV^* using equations 5.9, 5.5 and 5.13.

Question C.2g Show that the efficiency of the engine follows $\eta = \frac{\delta V^*}{|\ln(1-\delta V^*)|}$.

Question C.2h Plot the efficiency as a function of the relative step size and argue that small steps are more efficient.

From this analysis, it is clear that smaller engines are more efficient than larger ones. Indeed, the most efficient combustion motors have an efficiency of 50%, but the motors in cars that you find on the road usually only reach 20-30%. The molecular motors inside the cell have much higher efficiencies, as you calculated before, and can reach efficiencies close to 100%, just because they are tiny.

D. Test

1 Spheres in higher dimensions

Now you have seen how a Monte Carlo simulation works, and how the accuracy depends on the number of samples, we can employ it for a more daunting task: what is the surface area of an N-dimensional sphere? As an intermediate step, let's first calculate the volume of an N-dimensional sphere.

Question D.1a Write a function `calc_volume(n_dims,n=10000)` to compute the volume of a unit-sphere with radius 1, using a Monte Carlo algorithm. Test the function for `n_dims = 2` and 3. What does the volume of a 2-dimensional sphere represent?

Question D.1b Plot the volume of a unit-sphere for `n_dims` is 1 .. 15.

To obtain the surface area of the unit-sphere, you can compute the difference of the unit-sphere and a sphere with a radius that is `dr` smaller. This is the volume of a shell with thickness `dr`. To get to the area, you can divide this by its thickness.

Question D.1c Write a function `calc_surface(n_dims,dr=0.01,n=10000)` to compute the surface of a unit-sphere with radius 1, using a Monte Carlo algorithm. Test the function for `n_dims = 3`. Do you get the correct value?

Question D.1d Can you make `dr` arbitrarily small? Why (not)? What parameter in this calculation should you change to alleviate this effect?

Question D.1e Make a scatter plot of the area of a unit-sphere for `n_dims` is 1 .. 15. Plot the known values for the case of a 2D and a 3D sphere in the same figure with different colors. Optimize

the input parameters for the function `calc_surface()` to get a good match.

The analytical solution for the surface area of a unit sphere with n dimensions is:

$$A(n-1) = \frac{2 \pi^{n/2}}{\Gamma(n/2)} \quad (5.14)$$

with the Gamma function $\Gamma(n) = (n-1)!$. You can get the function `gamma` by importing it from the module `math`. Note that this `math.gamma()` function can handle non-integer numbers.

Question D.1f Reproduce the previous plot and add a continuous line plot of the analytical solution.

Monte Carlo simulations can help find numerical solutions for problems that are hard, or impossible, to solve analytically. They have proven quite helpful for computing coarse-grained structures of bio-molecules when molecular dynamics simulations require too much processing time.

2 The thermodynamic limit

In the derivation of the free energy, we separated the world into a small system and its environment, with the idea that fluctuations in the energy are small in the environment but may be large in the small compartment. But how small is small? To get a better idea, consider an isolated box that is split in the middle into compartments a and b with a thin non-permeable membrane. The membrane allows for the exchange of energy between the particles in both compartments, but there is no exchange of the particles themselves.

To calculate the entropy in each of the compartments you can use the Sackur-Tetrode equation (eq. 5.5) and put all fixed parameters together in a constant. This yields for compartment a : $S(E_a) = 3/2 k_B N_a \ln(E_a) + \text{const}$. Looking carefully, this formula is a bit odd: the parameter within the logarithm has a unit, which is not allowed. However, the constant in the end gives us the freedom to multiply or divide the parameter E_a by any parameter and adjust the arbitrary constant such that the outcome is the same. So, if we don't care about the offset in the entropy (and we don't because generally, we are only interested in changes in entropy, not its absolute number), we can express E_a in any unit as long as we use the same unit if we want to compare results.

Question D.2a Write a function `calc_entropy(E,N)` that yields the entropy. You may leave the constant out, as it is arbitrary anyway. Get the numerical value of k_B from the module `scipy.constants`.)

We want to compare what happens with the entropy in both compartments as the total energy in the system is distributed between compartments a and b . By the same virtue, as being allowed to input the energy in any unit, we can just express E_a as the fraction of the total energy in the micro-canonical system.

- Question D.2b** Use the function `calc_entropy(E,N)` to plot in the same figure the entropy in compartment a , compartment b and the total entropy for $0 < E_a/E_{total} < 1$ and $N_{total} = 10$. Assume that the particles are equally distributed between both compartments. Plot the entropy in units of k_B to eliminate very large numbers.
- Question D.2c** Plot the probability density function of E_a as a function of E_a/E_{total} .
(Hint: Remember that the entropy is related to the number of ways to distribute the energy and that each particular way to distribute the energy is equally likely.)
- Question D.2d** Iterate the calculation and plot the probability density function for $N_{total} = 10, 100, 1000$ and 10000 .
(Tip: You may run into numerical problems when the value of the entropy is too small. Use its arbitrary offset to avoid this problem).
- Question D.2e** Use the central moments to calculate the standard deviation of E_a/E_{total} for a range of N_{total} of at least 5 orders of magnitude. Divide your result by E_a/E_{total} for $N_{total} \rightarrow 1$ (which equals the thermal energy of a single particle and takes care of the hitherto unspecified temperature). Plot the results on a log-log scale as a scatter plot. Do the fluctuations scale again with $1/\sqrt{N}$? Plot this relation as a line in the same figure.
(Tip: You may need to refine the accuracy with which you vary E_a/E_{total} , such that your pdf has enough resolution to show the variations.)
- Question D.2f** The thermodynamic limit is the limit in which statistical fluctuations of the system parameters are negligible. What is the minimal number of particles in the system if we set the threshold of the fluctuations at 0.1% of that of a single particle?

Chapter 6

Reactions

A. Background

In Chapter 5 you learned that the universal second law of thermodynamics drives all systems towards an equilibrium state that is characterized by the highest possible entropy. Biological systems have evolved to partially avoid (or at least delay) this outcome by coupling two processes, in which one may decrease entropy, but only if the second simultaneous process increases entropy, such that the net effect obeys the second law. A prominent example of this strategy is photosynthesis: the sun light that falls on earth may just heat up the planet (a low free energy state), but plants have the machinery to use some of the absorbed light to convert water and carbon dioxide to oxygen and sugar (a high free energy state). In the end, this sugar and oxygen will break down again to water and carbon dioxide, but in between it takes part in many life processes that give our planet its highly diverse ecosystem.

It is important to note that all living systems are not in thermodynamic equilibrium, though some sub-processes may be. This shifts the question from 'what is the state of highest entropy (or equivalently lowest free energy)?' to what 'determines the rates of the processes towards this state?'

It should also be clear by now that chemical reactions play a central role in many of life's processes. This is not only because chemical bonds provide structure to (bio-)molecules, they may also contain a lot of energy, that can be harnessed to power reactions that by themselves increase free energy. In this chapter, we will look into reaction rates and how enzymes can catalyze reactions, such that the processes in the cell can be regulated. We will aim for a mechanistic insight. Though ultimately it takes quantum mechanics to describe (the making/breaking of) chemical bonds, you will here use a more coarse-grained approach to get the bigger picture of the answer to this magnificent question:

If all processes yield the highest entropy state, how can a cell control these processes such that they occur in the correct order?

B. Theory

In this section you will:

- relate the partition function of a complex system to its free energy
- learn how conformational changes are the key to understanding reaction rates
- see how two reactions can be coupled to drive a process in a defined direction

1 Complex systems

The formalism of minimizing the free energy was very useful in reducing the complexity of a canonical thermodynamic system (a system that features free exchange of energy with its environment) into a description in which the entropy of the environment could be captured in a single constant: temperature. Still, it's a hideous task to keep track of all coordinates and all velocities of all atoms of even a very small system, like for example a single protein. Optimized molecular dynamics simulations on state-of-the-art computers still only manage to simulate the behaviour of protein-sized ensembles for times smaller than 1 millisecond. This has long been a bottleneck in understanding proteins and engineering drugs to manipulate them, though recent advances in machine learning have made it possible to make some shortcuts and still obtain proper structures.

However, there is another way out: most of the variations in structure are minute, and large coordinated transitions of the protein parts occur rarely. This makes it much more efficient to group closely related structures into complex states that contain a large number of very similar structures. For such an ensemble of N similar states the average energy will be:

$$\langle E \rangle = \sum_{i=1}^N E_i p_i \quad (6.1)$$

where the probability p_i of being in state i is defined by Boltzmann (eq. 3.4). The entropy within this ensemble can be obtained by combining equations 5.1, 5.4 and 5.3. Together, they define the free energy for the ensemble as:

$$G \equiv \langle E \rangle - TS = -k_B T \ln Z \quad (6.2)$$

in which Z is the partition function that was used to normalize the Boltzmann probabilities (eq. 3.5). (Note that we used the Gibbs free energy rather than the Helmholtz free energy, as is commonly done and is allowed when volume changes are negligible)

Question B.1a Derive the free energy of an ensemble of closely related conformations, as stated in eq. 6.2.

Question B.1b Calculate the change in volume that would generate an amount for work of $0.1 k_B T$ at atmospheric pressure. Express this volume change in nm^3 for easy reference.

Now imagine that all N conformations of the protein can be grouped in 2 distinct ensembles, with n complex states s_A and m complex states s_B . An example of a protein that has two distinct structures is shown in figure 6.1, but many different proteins feature similar conformation changes. The probability to be in state s_A becomes:

$$P(s_A) = \frac{1}{Z} \sum_{i=1}^n \exp\left(\frac{-E_i}{k_B T}\right) \quad (6.3)$$

Using a derivation very similar to the that used to derive eq. 6.2 it is possible to obtain a Boltzmann-like equation, in which the energy is replaced by the free energy, taking care of the number of different conformations in each complex state:

$$\frac{[s_B]}{[s_A]} = \frac{P(s_B)}{P(s_A)} = \exp\left(\frac{-\Delta G}{k_B T}\right) \quad (6.4)$$

in which the square brackets indicate the concentration. Equation 6.4 uses the entropy within the complex state to take into account the large number of slightly different states in ensemble A : a large number of very similar conformations will increase the probability of that complex state as it decreases the free energy. Thus, it is not required to know the number of different conformations that contribute to a complex state.

Of course, the protein cannot change conformations instantaneously. There will be intermediate conformations in which sub-units of the protein in structure A bend toward the location of those sub-units in structure B . For adenylate kinase, the protein depicted in figure 6.1, the red and the yellow sub-units bend towards each other, closing the gap in between them. To characterize how far the transition has progressed it is useful to define a reaction coordinate. This can be any measurable parameter that quantifies how far the transition is. In the case of adenylate kinase, the distance between the mean position of the yellow sub-unit and the mean position of the red sub-unit would be a good choice. (Alternatively, the distance between one amino acid from each sub-unit would also be fine, though a bit more noisy).

Independent of the precise choice of reaction coordinate, it is clear from the open structure that the two sub-units have some freedom to move around, because of the flexible link to the blue protein core. Figure 6.1C shows a schematic plot of the free energy of the protein as a function of the reaction coordinate. The high flexibility is reflected by the shallow free energy profile: it costs little energy to bend the two sub-units from the most favorable conformation shown in 6.1A, to a similar, but somewhat more closed conformation. Here, we have used a harmonic potential, in which the free energy increases quadratically with the reaction coordinate. In reality, the energy landscape can be much more rugged, but fluctuations of order $k_B T$ are easily overcome. In the closed conformation, figure 6.1B, there may be more interactions with the blue core, which not only stabilizes this conformation but also makes the structure more rigid. This results

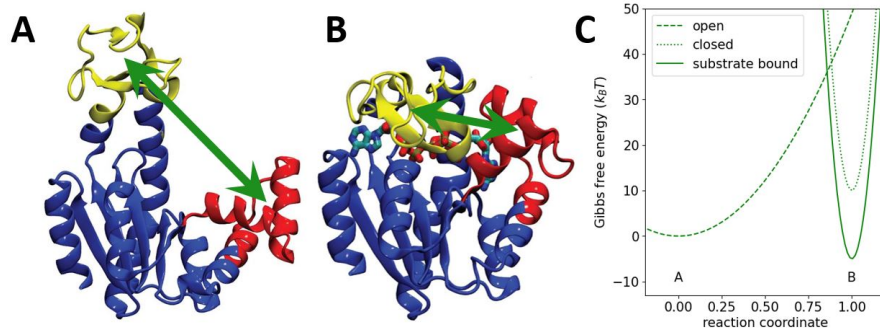


Figure 6.1: Two distinct conformations of the protein adenylate kinase. This protein is a phosphotransferase enzyme that catalyzes the inter-conversion of the various adenosine phosphates (ATP, ADP, and AMP). By constantly controlling phosphate nucleotide levels inside the cell, adenylate kinase plays an important role in cellular energy homeostasis. A) The open conformation, featuring two sub-units that are flexibly connected to the blue core of the protein. B) The closed conformation, which contains the substrate adenosine phosphates (in cyan, but barely visible) clamped by the two flexible sub-units that moved towards each other. The green arrows indicate the distance between the two sub-units and could be used as a 'reaction coordinate' C) Schematic plot of the Gibbs free energy of the open, closed, and substrate-bound conformations as a function of the distance between the sub-units. (A and B: modified image from Oradd et al. 2021 Science Advances)

in a much narrower potential for this conformation. The minimum free energy of state *B* may be larger than that of state *A*, if only because of the stress in bending both sub-units into the clamp.

Question B.1c Use figure 6.1C to estimate the ratio between the concentrations of the lowest energy open and the lowest energy closed conformation in equilibrium.
(Tip: Ignore the substrate-bound state for this question.)

Whereas the ratio between the concentrations of the two states is defined by their differences in free energy, the rate at which this equilibrium is established requires further analysis. Keep in mind that the equilibrium is 'quasi-static' in the sense that the concentrations will not change. However, there is a continuous transition from proteins in state *A* to state *B*, and also in the reverse direction:



in which k_+ indicates the rate of the forward reaction and k_- the rate of the reverse reaction. Given these rates, the change in the number of molecules in states *A* and *B* will be:

$$\begin{aligned} \frac{dN_A}{dt} &= -k_+ N_A(t) + k_- N_B(t), \\ \frac{dN_B}{dt} &= k_+ N_A(t) - k_- N_B(t) \end{aligned} \quad (6.6)$$

Question B.1d Use $N_{tot} = N_A + N_B$ to eliminate N_B from the first equation in eq. 6.6.

Question B.1e Express the number of molecules in state A at equilibrium, $N_{A,eq}$ in terms of k_+ , k_- and N_{tot} .

Question B.1f Write down a new differential equation $d(N_A - N_{A,eq})/dt$, based on eq. 6.6, and show that the number of molecules in state A starting at $N_A(0)$ will reach $N_{A,eq}$ according to:

$$N_A(t) = N_A(0) + (N_A(0) - N_{A,eq})e^{-(k_+ + k_-)t} \quad (6.7)$$

The number of molecules in state B will change at the same rate. Equation 6.7 states that concentrations will reach equilibrium exponentially, but can we learn more about the time constant? For this, the energy diagram in figure 6.1 is also informative. From the figure, it is clear that the sub-units need to move along the reaction coordinate from state A to state B . Along the way, the free energy increases up to the point where the two energy curves cross. At this point, it is unclear whether the sub-units bent inward from state A , or bent outward from state B . We'll call this state the transition state. At the transition state, the molecule can decrease its free energy by either falling back to state A or by moving forward to state B . Both events can occur with equal probability, as there is no memory in the system (inertia doesn't play a role at the low Reynolds number for this movement). So the rate is proportional to how frequently the molecule enters the transition state. How often the molecule is in the transition state, relative to finding it in state A can also be calculated by eq. 6.4, by just plugging in the energy of the transition state $\Delta G^\ddagger$.

Question B.1g How can you extract $\Delta G^\ddagger$ from figure 6.1C? Use the figure to explain which transition will be faster: $A \rightarrow B$ or $B \rightarrow A$?

Question B.1h Show that the ratio's of the rates is constant and is equal to:

$$\frac{k_+}{k_-} = K = \exp(-\Delta G/k_B T) \quad (6.8)$$

The equilibrium constant K defines the ratio between the species A and B and will be independent of the starting condition or the total amount of both species. The absolute value of the rates will depend on how fast the sub-units move in the potential well. This is again a diffusion problem, but instead of free diffusion in a box, the sub-unit is bound by a flexible tether that will pull it back. The parabolic energy profile suggests that the bending of the sub-unit follows a Hookean response, characterized by a spring constant κ .

Question B.1i What is the potential energy of a Hookean spring? If the unit of the reaction coordinate in figure 6.1C is nanometer (which is approximately correct), estimate the spring-constant κ .

The time constant for relaxation of a Hookean spring in a viscous environment follows:

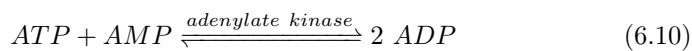
$$\tau = \xi/\kappa \quad (6.9)$$

in which ξ represents the Stokes drag coefficient.

Question B.1j Check the units to verify this equation. Use figure 6.1 to estimate the time constant τ .

2 The chemistry

In chapter 5 you have seen that it is possible to allow for processes that increase the free energy if, and only if, this process is coupled to another one that lowers the free energy, such that the net result is a reduction of the free energy. Here we will look into the details of the reaction catalyzed by the enzyme adenylate kinase, but the general idea of the physics and chemistry behind this reaction is very similar in all enzyme-mediated reactions. In the case of adenylate kinase, the process that reduces the free energy is the following reaction:



in which *ATP* represents Adenosine-Tri-Phosphate, *AMP* Adenosine-Mono-Phosphate and *ADP* Adenosine-Di-Phosphate. The equilibrium constant for this reaction is defined by:

$$K = \frac{[ADP]^2}{[ATP][AMP]} \quad (6.11)$$

Actually, the energy contained in 1 molecule *ATP* and 1 molecule *AMP* is the same as the energy in 2 molecules of *ADP*, which would yield an equilibrium constant $K = 1$. Still, the reaction that forms *ADP* is much faster than the reverse in vivo. This must be because of the entropy part in the free energy. In vivo, the concentration of *ATP* is 8 times larger than that of *ADP* and 100 times larger than that of *AMP*.

Question B.2a What is the free energy difference of the reaction under these conditions?

Question B.2b The same enzyme can also accelerate the reverse reaction. What is needed to make the reverse reaction faster than the forward reaction?

Reaction 6.10 is a chemical equilibrium, in the sense that a chemical bond is broken and a new one is made (as opposed to reaction 6.5, which is a mechanical equilibrium). Without the enzyme adenylate kinase, the reaction would still occur spontaneously, but it takes a very long time. The schematical drawing of

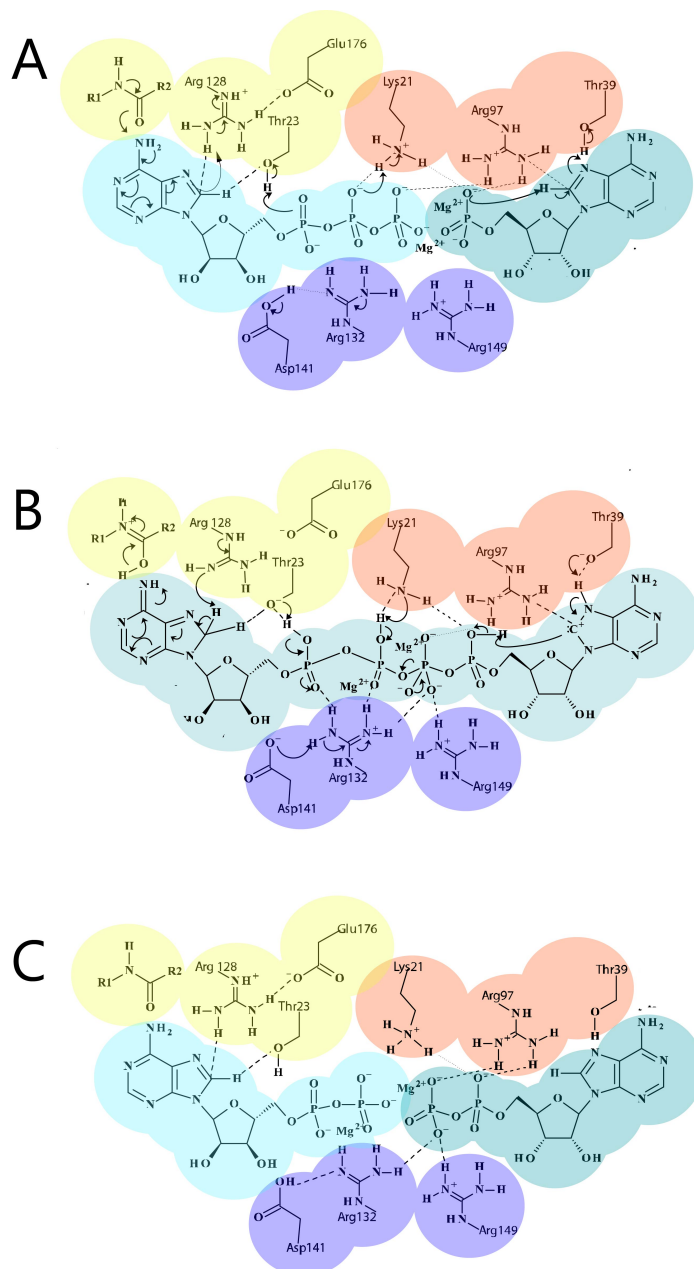


Figure 6.2: Three snapshots of the reaction pathway of adenylate kinase in action. The color scheme is the same as in figure 6.1A and B. Adenosine phosphates are colored in different shades of cyan. Solid lines between atoms represent chemical bonds, dashed lines indicate hydrogen bonds and arrows indicate rearrangements of electrons. Note that only the amino acids that play a role in the reaction are drawn. (Modified from Kenyon and Roth 2012 Biochemistry)

the energy landscape as a function of the progress of the reaction has a similar shape as plotted in figure 6.2C, but the reaction coordinate represents a different parameter and the energy barrier is actually much higher.

The function of the adenylate kinase is to lower the activation energy of the reaction. It provides an alternative reaction pathway, which avoids high energy states. The way it manages to do this is by mediating the simultaneous transfer of several electrons at different locations. Figure 6.2 shows the detailed chemistry. There is no need to go too deeply into the detailed organic chemistry of the figure, but it is worthwhile to highlight some aspects that relate to the physics of the enzyme.

Question B.2c There are 2 *ADP*'s, 1 *ATP* and 1 *AMP* in the reaction scheme. Indicate for each in which sub-figure they are drawn (A, B, or C). Argue why one of the sub-figures has none of the three.

Question B.2d Has the protein changed between figure 6.2 A and C. Why (not)?

The small curved arrows indicate the reallocation of an electron to a different atom. Note that such transfer of electrons between atoms can only occur very locally, i.e. within several 0.1 nm.

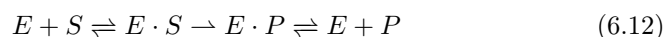
Question B.2e What are the implications of this for the structure of the protein?

Question B.2f Sometimes the metaphor of a key and a lock is used in relation to an enzyme catalyzed reaction. Explain the resemblance.

Question B.2g What would happen if one of the amino acids that participate in the reaction would be replaced by another amino acid, for example, due to a mutation in the gene encoding for the enzyme?

Question B.2h Explain how a mutation of amino acids that are not directly involved in the reaction can still affect the reaction rate.

The combination of an enzyme *E* catalyzed structural and chemical reaction from substrate *S* to product *P* can be summarized in the following reaction scheme:



in which the single arrow in the middle reflects the chemical reaction that yields a large reduction in free energy, driving the reaction to the right. In the case of adenylate kinase, a mechanical movement of the protein is required to boost the rate of a chemical reaction that is important for the cell.

You have also seen examples of reverse functionality: the enzyme kinesin executes a mechanical change of its structure, such that it translocates over microtubules and transports cargo. The only purpose of the coupled chemical reaction ($ATP \rightleftharpoons ADP + P_i$) is to provide enough of a change in free energy to allow this to happen. Still, kinesin is just another enzyme that catalyzes a chemical reaction. Whether the purpose of the enzyme is to generate motion or to catalyze a chemical reaction doesn't matter: the reaction mechanism is very similar.

C. Exercises

In this section you will:

- visualize how the entropy within a complex system affects the distribution of complex states
- advance Monte Carlo simulations to probe complex energy landscapes
- simulate conformational changes in single proteins

1 Entropy in complex states

In this exercise, you will advance the Monte Carlo scheme to simulate diffusion in a potential well. Let's start with the potential that combines the open and the closed state of adenylate kinase, as displayed in figure 6.1.

To keep the code clean, it is better to put all the parameters that define this potential in a single dictionary that looks like: `potential={'delta_x':1,'k1':1000,'k2':1000,'delta_e':0}`. Note that only the relative shift and height of the second energy well need to be known. The parameters are expressed in units of nm , pN/nm and $k_B T$.

Question C.1a

Create a function `calc_energy(x,potential)` that yields the lowest free energy of the two states, given the reaction coordinate `x`. Plot the energy and choose parameters that reproduce the non-substrate-bound potential in figure 6.1c. Note that this figure uses a combination of two harmonic potentials.

(Tip: use the `numpy` function `amin()` to obtain the minimum along one axis of a 2-dimensional array.)

Analogous to the Monte Carlo simulations in the previous chapter, you can feed a large number of randomly chosen distances into the function `calc_energy()` and use the Boltzmann equation (eq. 3.4) to compute the probability to be at a certain reaction coordinate. However, as opposed to the problem of finding a value for π , where the coordinates were bound by the unit sphere and its encapsulating square, the potential is not bound, so a priori it is not known what range of coordinates you should use.

In statistics, the 68–95–99.7 rule (see for example here) says that 99.7 percent of the samples should be within 3 standard deviations of the mean for a normal

distribution. You can apply this criterion separately to both harmonic wells in the combined potential.

Question C.1b What minimal and maximal value of the reaction coordinate should you choose according to this criterion?
(Hint: use the Boltzmann equation.)

You can probe the relative abundance of each position using a Monte Carlo scheme: just generate a large array with random, uniformly chosen positions. Then calculate the Boltzmann weight, given the potential, and normalize it. Use the parameters `delta_x=1, k1=100, k2=1000, delta_e=1`.

Question C.1c Make a large numpy array with random, uniformly chosen positions within the appropriate range and make a scatter plot of the probability to be at those positions according to Boltzmann (eq. 3.4) as a function of the position.
(Tip: don't forget to normalize!)

Note that you did not obtain the probability function, but rather the outcome of a randomly sampled data set of that function. The probability of being in state 2 is still the part of the area under the curve that corresponds to state 2, but because it is sampled randomly it is not trivial to compute $\int_{s_2} P(x)dx$.

Question C.1d Estimate 'by eye' the ratio $P(s_1)/P(s_2)$ and explain why it is not equal to $e^{-E_1/k_B T}/e^{-E_2/k_B T}$. Include the stiffness of the states and their entropy in your answer.

2 Metropolis Monte Carlo

Because the coordinates in the above Monte Carlo simulation are chosen randomly, it's impossible to extract the rates from the simulation. To probe the kinetics of the transition we need to make an adjustment that was first proposed by Nicholas Metropolis in 1953. Rather than probing the energy landscape at random fixed positions, the Metropolis Monte Carlo algorithm uses a random walker along the energy landscape. Like in the case of the Galton board, one takes small equally sized steps in a random direction. If this was the only modification, it would yield the same result as the algorithm in question 6C.1.

However, in the Metropolis Monte Carlo algorithm, the probabilities of the random step depend on the energy landscape. Steps that lower the energy are always accepted, but steps that increase the energy are accepted with a probability that decreases with the energy increase, following a Boltzmann-like criterion:

$$accept = \exp(-\Delta G/k_B T) > random[0,1] \quad (6.13)$$

Question C.2a Explain that steps that decrease the energy are always accepted using the Metropolis criterion.

Question C.2b What would happen if all steps that increase the energy are rejected?

So, whereas the Galton board-type of random walker has a fixed probability of the direction of the steps (which is not necessarily 0.5!), in the Metropolis scheme the probability depends on the position in the energy landscape. This also means that the steps are not independent anymore and one cannot compute successive steps in a parallel fashion. Thus, a `for`-loop in the code can not be avoided.

The entire Metropolis Monte Carlo algorithm will be as follows:

- create an array `steps` that contains `nsteps` randomly chosen values from `[-stepsize, stepsize]`
- create an array `accept_probability` that contains `nsteps` randomly chosen values distributed uniformly between 0 and 1
- create an array `position` that contains a single element: the start position
- iterate through `steps` and `accept_probability` and append the new position (`= lastposition + step`) only if the Metropolis criterion is `True`

Question C.2c Implement the function `metropolis(nsteps, stepsize, x0, potential)` accordingly. Make sure that the output is a `numpy` array with the positions of all accepted steps.

First, you will examine the results of diffusion in a single potential well. For this you can still use your function `calc_energy()`, but make sure that the second state is inaccessible by setting `delta_x=0, delta_e=10` and `k1=k2=10`.

Question C.2d Plot the energy as a function of position for a range corresponding to energy differences up to $10 k_B T$ and check visually that you indeed have a single harmonic potential.

Question C.2e Test the function `metropolis()` using this potential by plotting the positions after 10000 iterations. Is the result what you expect? Why (not)?

Question C.2f Plot the probability density function of the obtained positions as a scatter plot and plot the expected theoretical curve (based on the potential) as a line. Does the Metropolis algorithm produce the expected result?

Question C.2g Iterate the function 1000 times for `nsteps=2000, stepsize=0.02` and `x0=0`. Plot the mean squared displacement of the first 1500 steps. How does it compare with a freely diffusing particle?
(Note: the function `metropolis()` does not return the

same size array after each call (why?). Make sure that your code does not get stuck because of that.)
(Tip: you may want to use the function `tqdm()` in your for-loop to keep an eye on the progress.)

Question C.2h How can you extract the diffusion coefficient and the width of the well from this plot?

3 A rough landscape

Now that you have validated the Metropolis Monte Carlo algorithm, we can explore the dynamics of every energy landscape that you can think of. Let's explore the dynamics of a two-state system. You can copy and paste much of the code that you wrote in question 6C.2 to save time in answering these questions.

Question C.3a Find the parameters for the function `calculate_energy()` that creates a double well, spaced by 1 nm with equal energies and separated by an energy barrier of $1 k_B T$. Plot the energy landscape.

Before, we did not need to care too much about the step size. However, in the double well there is an additional length scale.

Question C.3b Why can the step size be too small? Why can the step size be too large? What is an optimal value?

Question C.3c Run the Metropolis Monte Carlo function on the double-well potential several times. Plot a time trace and describe what you see.
(Tip: try several step-sizes to confirm your previous answer)

Because the energy minima are the same in both wells, you would expect an equal number of instances of both states.

Question C.3d Make a scatter plot of the pdf of the simulated positions to verify this. Compare with a line plot for the theoretical value. If you did not find equal partitioning: what do you need to change in the simulation without changing the energy landscape?

Question C.3e Count the number of transitions from state 1 to state 2 for this potential and a sufficient number of steps. Find the parameters for a barrier of $4 k_B T$ and repeat the simulation. How many transitions do you find? What ratio do you expect?

Question C.3f Demonstrate the effect of introducing a difference in energy of $\Delta E = 2 k_B T$ between the wells. Plot the energy landscape, a representative time trace, and the pdf for the position.

D. Test

1 Motor proteins

The motor protein kinesin makes steps of 8 nm over a microtubule. The reaction mechanism is as follows: there is always one kinesin sub-unit bound to the track and to an ATP molecule. The lagging sub-unit may hydrolyze ATP and release the track. When the sub-unit has no bound ATP, it is relatively free to move. Upon binding of ATP, it will bind to the microtubule again on the plus-side of it, see figure 6.3. At that stage, when both sub-units are bound, the stalk that connects both 'feet' is much more restricted in its movement, which is reflected in a 10 times higher stiffness than in the unbound situation. There is a $5 k_B T$ activation barrier between the two states. In the second conformation, the ATP is hydrolyzed, which makes the second state $20 k_B T$ lower in energy than the starting conformation.

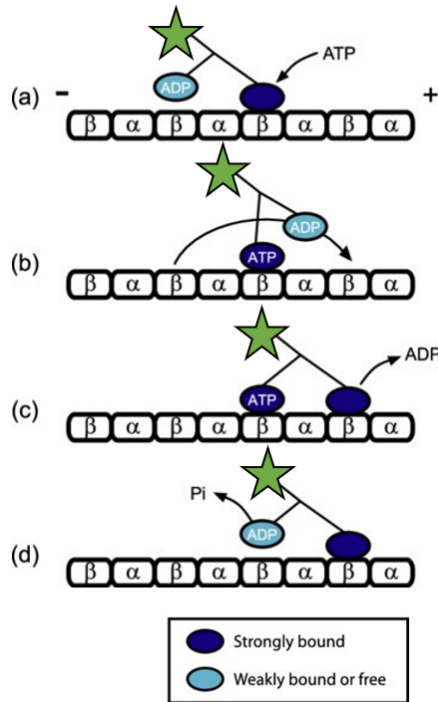


Figure 6.3: The processive motion of a kinesin motor protein follows a cycle of ATP-induced binding to the β units of the microtubule. The two 'feet'-subunits of kinesin, drawn as blue ovals, alternately hydrolyze ATP and while doing so release from the microtubule. The green star is the 'stalk' and depicts the progression of the motor. (Image modified from Nam et al. 2012 Journal of Physics: Condensed Matter)

Question D.1a Create a potential that quantitatively matches the description above and plot it.

Question D.1b Use the Metropolis Monte Carlo simulation to compute a time trace of a single step of the movement of the stalk. Choose appropriate settings for the simulation and plot a trace that displays a step forward.

After the power stroke has been executed and the ADP dissociates, the process can repeat for many cycles. This can be captured in a potential with periodic boundaries.

Question D.1c Adjust the function `calc_energy()` such that it makes a continuous potential with a period of 8 nm. Make sure that the bottom of the second well seamlessly connects to the bottom of the first well in the next step. Plot the potential for a few steps.
(Tip: make sure that the potential decreases by $20 k_B T$ for each step, so after 3 steps, the energy should be $-60 k_B T$.)

Question D.1d Run the simulation for long enough to observe at least 10 steps. Adjust the step size of the Metropolis Monte Carlo simulation (not that of the kinesin!), if necessary. Plot the result.

When the motor carries a load, there will be a viscous force working against the direction of translocation.

Question D.1e Include the force in the simulation by adding a work term to the energy. Do so by including a parameter `force` (in units of pN) in the dictionary `potential` and amend the function `calc_energy()`. Plot the resulting energy landscape for `force=[1,2,4]` pN.
(Tip: If you did not manage to create a periodic energy landscape, use the energy landscape of a single step.)

Question D.1f Run the simulation for `force=[1,2,4]` pN and plot the resulting trajectories. What is the effect of force on the average translocation rate, on the kinesin step size and the time between steps?

Question D.1g Can a force along the direction of translocation increase the translocation rate? Test this by doing a simulation with `force=[-2,2]` pN. Describe the result.